Consultas comprimidas sobre la Ecuación SAT

bit, rank, select y oráculos de conteo para CNF arbitrarias

Oscar Riveros
Santiago, RM, Chile
oscar.riveros@gmail.com

DeepSeek V4.1 Flash 1017 via Kilo Code
Colaborador de investigación (IA)
Proyecto Ariadna

Manuscrito de trabajo — 28 de octubre de 2026

Resumen

La Ecuación SAT asocia a una CNF F sobre n variables un entero $S(F)$ de 2^n bits cuya posición k vale 1 si y solo si la asignación de posición k falsifica F. Este artículo estudia cómo consultar $S(F)$ sin materializar sus 2^n bits y, sobre todo, qué parte de esa maquinaria puede acercarse al problema P frente a NP. La consulta puntual $\mathrm{bit}(k) = 1 - F(\mathrm{assign}(k))$ cuesta $O(m)$ evaluaciones de cláusula; las consultas agregadas rank, select, $\#\mathsf{SAT}$ y el muestreo uniforme concentran la dificultad. Se demuestra que una cobertura disjunta de la región prohibida equivale exactamente a una CNF equivalente a F con cubos falsificadores disjuntos (DF-CNF); que una DSOP de tamaño K responde rank en $O(nK)$ y select en $O(n^2K)$ sin materializar la tabla; que $\mathrm{rank}_0(k)$ es el número de modelos de F con $\mathrm{pos}(x) < k$, de modo que cualquier contador exacto produce un oráculo de rank y, por búsqueda binaria, de select y muestreo; y que un algoritmo híbrido usa la DSOP cuando es pequeña y el contador cuando no lo es. La novedad conceptual no es la codificación densa, sino el mapa de barreras: un compilador uniforme polinómico de cualquiera de estas representaciones implicaría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por el teorema de Toda, $\mathsf{PH} = \mathsf{P}$. El artículo corrige el teorema de consultas afines (las intersecciones incompatibles aportan cero, no el volumen de la pieza), incorpora el ejemplo de Sasao, discutido por Bernasconi, Ciriani, Luccio y Pagli, que descarta a la DSOP como ruta universal, y formula un programa explícito hacia $\mathsf{P} = \mathsf{NP}$: cotas inferiores para ADSOP, puertas traseras afines, compilación directa a ZDD y Fourier, y verificación de certificados. La instanciación subexponencial con los manuscritos de OpenAI y Anthropic (2026) se mantiene como la hipótesis H-ETH y se demuestra condicionalmente; no se usa como resultado establecido. La evaluación experimental, todavía preliminar, separa el coste de compilación del de consulta y confirma que el compilador, no el motor de consulta, es el cuello de botella.

Palabras clave: Ecuación SAT; COVERTRACE; compilación de conocimiento; subcubos disjuntos; rank/select; ZDD; conteo de modelos.

Abstract. The SAT Equation maps a CNF F over n variables to a 2^n-bit integer $S(F)$ whose k-th bit is 1 exactly when the assignment of position k falsifies F. This paper studies how to query $S(F)$ without materializing its 2^n bits, and how much of that machinery can be brought to bear on P versus NP. Point queries cost $O(m)$ clause evaluations; the aggregate queries rank, select, $\#\mathsf{SAT}$, and uniform sampling carry the difficulty. We prove that a disjoint cover of the forbidden region is exactly equivalent to a CNF equivalent to F with pairwise disjoint falsifying cubes (DF-CNF); that a size-K DSOP answers rank in $O(nK)$ and select in $O(n^2K)$ arithmetic operations without materializing the truth table; that $\mathrm{rank}_0(k)$ is the number of models of F with $\mathrm{pos}(x) < k$, so any exact counter yields a rank oracle and, by binary search, select and uniform sampling; and that a hybrid algorithm uses the DSOP when it is small and the counter otherwise. The conceptual contribution is not the dense encoding but the barrier map: a uniform polynomial compiler for any of these representations would imply $\#\mathsf{SAT} \in \mathsf{FP}$ and, by

Toda's theorem, $\mathsf{PH} = \mathsf{P}$. The paper corrects the affine-query theorem (incompatible intersections contribute zero, not the volume of the piece), incorporates the known Sasao example, as discussed by Bernasconi, Ciriani, Luccio, and Pagli, and shows that it rules out DSOP as a universal route, and states an explicit research program toward $\mathsf{P} = \mathsf{NP}$: lower bounds for ADSOP, affine backdoors, direct ZDD and Fourier compilation, and certificate verification. The subexponential instantiation with the OpenAI and Anthropic manuscripts (2026) is kept as hypothesis H-ETH and proved conditionally; it is not used as an established result. The experimental evaluation, still preliminary, separates compilation cost from query cost and confirms that the compiler, not the query engine, is the bottleneck.

Índice

1. Introducción **4**
1.1. Contribuciones 5
1.2. Organización 6

2. Preliminares **6**
2.1. El cubo booleano y las CNF 6
2.2. La Ecuación SAT generalizada 7
2.3. DSOP y DF-CNF 7
2.4. Consultas sobre el bitvector 7
2.5. Compiladores, certificados y uniformidad 8
2.6. Modelo de complejidad 8

3. Consultas puntuales **8**

4. La forma generalizada y las DF-CNF **9**
4.1. La Ecuación SAT para CNF arbitrarias 9
4.2. DF-CNF: la dualidad exacta 10

5. Rank y select desde una DSOP **11**
5.1. Conteo de prefijos en un cubo 11
5.2. El motor DSOP 12
5.3. De la CNF a la DSOP 13

6. Un oráculo de conteo para rank y select **13**
6.1. El comparador de prefijos 13
6.2. Reducción de rank a conteo exacto 13
6.3. La instanciación H-ETH 14

7. La frontera de complejidad: mapa de barreras y programa hacia $\mathsf{P} = \mathsf{NP}$ **14**
7.1. La condición de colapso 14
7.2. La rama DSOP: el certificado y su tamaño 15
7.3. La rama de conteo: $\#\mathsf{SAT}$ y la dirección de la reducción 15
7.4. La rama H-ETH: subexponencial no es polinómico 16
7.5. La rama de Fourier 16

7.6. La rama ZDD y la no uniformidad . . . 16
7.7. La rama estocástica y la rama afín . . . 17
7.8. Resumen de las rutas . . . 17
7.9. Certificados, verificación y complejidad de pruebas . . . 18
7.10. Islas de tratabilidad y la pregunta de cobertura . . . 18
7.11. Las islas no son una clasificación vacía . . . 19
7.12. Paralelización y el modelo de coste . . . 19
7.13. Qué se necesitaría para un colapso . . . 21
7.14. Programa hacia $\mathsf{P} = \mathsf{NP}$ . . . 21

8. Algoritmo híbrido **22**
8.1. Qué garantiza el tope $K_{\max}$ . . . 23
8.2. Por qué el híbrido no es inevitablemente exponencial . . . 23
8.3. Condiciones de éxito por rama . . . 24

9. Optimización mediante ZDD y Fourier **26**
9.1. Representación de Fourier . . . 26

10. Oráculos estocásticos y coberturas afines **27**
10.1. Tipos de oráculo y separaciones . . . 27
10.2. Muestreo, hashing y la cancelación de Karp–Luby . . . 28
10.3. Cobertura afín: ADSOP . . . 30
10.4. Compilador ADSOP directo . . . 33
10.5. Límites y problemas abiertos . . . 36

11. Protocolo experimental **37**
11.1. Instancias . . . 37
11.2. Métricas . . . 37
11.3. Criterios de aceptación . . . 37
11.4. Hipótesis experimentales . . . 37
11.5. Protocolo para las alternativas . . . 37
11.6. Protocolo para el híbrido . . . 38
11.7. Tres escenarios de medición . . . 39
11.8. Resultados preliminares: DSOP frente a Kerberos . . . 39

12. Trabajo relacionado **41**

13. Discusión y problemas abiertos **43**
13.1. Resultados incondicionales . . . 43
13.2. El límite estructural . . . 43
13.3. Problemas abiertos . . . 43
13.4. Alternativas y garantías . . . 44
13.5. Relación con la práctica . . . 44

14. Conclusiones **45**

1. Introducción

La Ecuación SAT, introducida en [1] para CNF balanceadas y generalizada en [2] a CNF arbitrarias, codifica la tabla de verdad de una fórmula en un único entero. Para una CNF F sobre n variables, $S(F)$ tiene 2^n bits; el bit en la posición $\mathrm{pos}(x)$ vale 1 si y solo si x falsifica F. La nota unificadora [4] demostró que este entero y la compilación COVERTRACE de [3] representan el mismo objeto: la región prohibida $U(F) \subseteq \Omega_n$.

La representación es exacta, pero materializarla cuesta 2^n bits. La pregunta de este artículo es cómo consultarla sin construirla. Conviene separar dos nociones. La *consulta puntual* decide el bit $\mathrm{bit}(k)$ de una posición dada; para la Ecuación SAT es directa, porque $\mathrm{bit}(k) = 1 - F(\mathrm{assign}(k))$, donde $F(x) \in \{0,1\}$ es el valor de verdad de F en x. Cuesta $O(m)$ evaluaciones de cláusula, el coste de evaluar la CNF. Las *consultas agregadas* cuentan unos o ceros en un prefijo (rank), localizan el r-ésimo uno o cero (select), cuentan el total (#SAT) o muestrean uniformemente. No se responden evaluando una sola asignación; son las que codifican la dificultad de contar modelos.

La tesis del artículo es que la forma generalizada de la Ecuación SAT no solo es una representación densa de la tabla de verdad, sino la interfaz natural para compilar estructuras de consulta sobre ella. La DSOP de COVERTRACE proporciona una representación dispersa que, cuando es pequeña, responde rank y select en tiempo proporcional a su tamaño. Cuando la DSOP es grande, un contador exacto de modelos responde rank mediante la restricción de prefijo $\mathrm{pos}(x) < k$. El resultado es un motor híbrido que no materializa $S(F)$ y que usa cada representación en el régimen donde es competitiva.

El artículo persigue dos objetivos que conviene no mezclar. El primero es técnico: construir y analizar la capa de consulta. El segundo es estratégico: determinar qué parte de esa capa puede contribuir a una prueba de $\mathsf{P} = \mathsf{NP}$ y qué parte no. La respuesta corta es que la capa de consulta no es el problema; el problema es la compilación. El artículo convierte esa observación en un mapa de barreras y en un programa de investigación, no en una demostración de colapso.

El artículo también explora dos alternativas a la compilación clásica por cubos. La primera es estocástica: el muestreo directo, el hashing XOR y la estimación de la DNF prohibida de Karp–Luby. La segunda es algebraica: una cobertura por subespacios afines sobre $\mathbb{F}_2$, que contiene a la paridad en una sola pieza y admite conteo, rango y selección exactos. La Sección 10 presenta ambas, demuestra la corrección de las consultas afines y describe un prototipo de compilador ADSOP directo que usa un solver SAT como oráculo.

El artículo separa dos niveles de resultado. Los teoremas de las Secciones 4–6 son incondicionales —la equivalencia DSOP–DF-CNF, el motor rank–select y la reducción de rank a conteo exacto—, con la excepción de la instanciación subexponencial del Apéndice A, que es condicional a H-ETH. La Sección 7 analiza la consecuencia más delicada: por qué el motor no implica $\mathsf{P} = \mathsf{NP}$. La razón es estructural: hay una separación entre el coste de consultar una representación y el coste de construirla, y el artículo demuestra exactamente qué condición adicional haría falta para un colapso.

Hay además un resultado que cambia el mapa y que el artículo incorpora como pieza propia: la ruta DSOP no puede ser universal. Sasao [25] exhibió una función con una SOP de $n/2$ términos cuya DSOP tiene $2^{n/2} - 1$ cubos; Bernasconi, Ciriani, Luccio y Pagli [26] discuten ese ejemplo como muestra de que el crecimiento exponencial persiste para soluciones mínimas. Dualizando la construcción se obtiene una CNF con $O(n)$ cláusulas y $\chi_{\sqcup}(U(F)) = 2^{\Omega(n)}$ (Proposición 4.8). Esto no demuestra $\mathsf{P} \neq \mathsf{NP}$; demuestra que una prueba de $\mathsf{P} = \mathsf{NP}$ no puede pasar por cubos disjuntos. Es el primer peldaño del programa: cerrar rutas antes de abrir otras.[1]

[1] Una versión anterior de este manuscrito trataba la separación DSOP como un problema abierto. La lectura

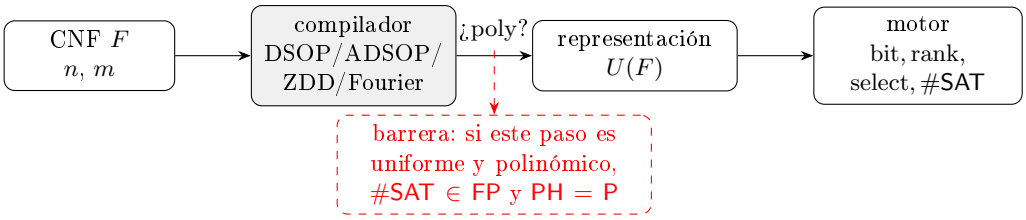


Figura 1: El motor separa la fase de compilación de la fase de consulta. La dificultad de complejidad está en la primera; la segunda es la que el artículo resuelve.

1.1. Contribuciones

C1. Modelo de consultas. Se formalizan bit, rank, select, #SAT y muestreo uniforme sobre el bitvector $S(F)$ (Sección 2).

C2. Ecuación generalizada y DF-CNF. Se demuestra que una cobertura disjunta de $U(F)$ equivale exactamente a una CNF equivalente a F cuyos cubos falsificadores son disjuntos. Esto convierte la dualidad acarreo–disyunción de [4] en una igualdad de mínimos y justifica que la salida de COVERTRACE sea una DF-CNF (Sección 4).

C3. Rank/select desde una DSOP. Si $\mathcal{U}$ es una cobertura disjunta de tamaño K, entonces $\mathrm{rank}_1(k)$ se calcula en $O(nK)$ operaciones aritméticas, $\mathrm{select}_0(r)$ en $O(n^2K)$, #SAT en $O(K)$ y el muestreo uniforme en $O(n^2K)$ (Sección 5).

C4. Oráculo de conteo. Se demuestra que $\mathrm{rank}_0(k) = \#\mathsf{SAT}(F \wedge [\mathrm{pos}(x) < k])$. Cualquier contador exacto de modelos produce un oráculo de rank; con $O(n)$ consultas se obtiene select y muestreo. La instanciación con los manuscritos de OpenAI [5] y Anthropic [6] se formula como la hipótesis H-ETH (Apéndice A).

C5. Algoritmo híbrido y protocolo experimental. Se combinan DSOP y oráculo de conteo, se analiza el coste y se propone un protocolo reproducible (Secciones 8 y 11).

C6. Representación de Fourier. Se formaliza el espectro de Walsh–Hadamard de $S(F)$; un espectro de tamaño s responde rank en $O(sn)$ y select en $O(sn^2)$. La paridad tiene $s = 2$ y DSOP 2^{n-1}; una cláusula ancha tiene $s = 2^w$ y DSOP 1. La construcción de un espectro polinómico para toda CNF es una ruta al colapso y un problema abierto (Sección 9.1).

C7. Separación de la ruta DSOP. Se incorpora la construcción de Sasao [25], discutida por Bernasconi, Ciriani, Luccio y Pagli [26], y se demuestra que existe una familia de CNF con $O(n)$ cláusulas y $\chi_{\sqcup}(U(F)) = 2^{\Omega(n)}$. La DSOP no puede ser la representación universal; cualquier prueba de $\mathsf{P} = \mathsf{NP}$ tiene que ir más allá de los cubos disjuntos (Proposición 4.8).

C8. Frontera de complejidad y mapa de barreras. Se demuestra que un compilador uniforme polinómico de DSOP implicaría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por el teorema de Toda, $\mathsf{PH} = \mathsf{P}$; la misma implicación se extiende a ADSOP, ZDD, espectro de Fourier y a un contador exacto polinómico. El motor no proporciona ninguna de esas piezas: sus cotas polinómicas son condicionales al certificado o al contador. Se formalizan las nociones de compilador uniforme y certificado, se definen las islas de tratabilidad y se formula un programa explícito hacia $\mathsf{P} = \mathsf{NP}$ (Sección 7).

C9. Corrección y validación de ADSOP. Se corrige la fórmula de intersección de un patrón con una pieza afín: una intersección incompatible aporta cero, no el volumen de la pieza. La corrección se propaga a la demostración, al algoritmo y al protocolo experimental, que incluye intersecciones vacías, intersecciones de dimensión cero y sistemas compatibles de rango completo (Secciones 10.3 y 11).

C10. Alternativas estocásticas y afines. Se analizan oráculos de muestreo y hashing, la cancelación del complemento en Karp–Luby y una cobertura por subespacios afines (ADSOP) que responde consultas exactas. Se presenta un compilador ADSOP directo basado en SAT

de [26] y de la fuente original [25] obligó a corregir esa afirmación; el resultado se recoge en la Proposición 4.8.

y se valida en $n \le 10$; no es un compilador polinómico (Sección 10).

1.2. Organización

La Sección 2 fija notación, el modelo de consultas y la distinción entre compilador y certificado. La Sección 3 trata las consultas puntuales. La Sección 4 demuestra la equivalencia DF-CNF y la separación de la ruta DSOP. La Sección 5 construye el motor rank–select desde una DSOP. La Sección 6 reduce rank a conteo exacto. La Sección 7 presenta el mapa de barreras, las islas de tratabilidad y el programa hacia $\mathsf{P} = \mathsf{NP}$; es la sección que organiza la discusión de complejidad. La Sección 8 presenta el algoritmo híbrido. La Sección 9 discute las optimizaciones con ZDD y Fourier. La Sección 10 explora oráculos estocásticos y coberturas afines. La Sección 11 propone el protocolo experimental. Las Secciones 12 y 13 sitúan el trabajo, y la Sección 14 cierra.

2. Preliminares

2.1. El cubo booleano y las CNF

Sea $\Omega_n = \{0,1\}^n$. Para $x = (x_1, \dots, x_n) \in \Omega_n$, la *posición* de x es

$$\mathrm{pos}(x) = \sum_{i=1}^{n} x_i \, 2^{n-i} \in \{0, \dots, 2^n - 1\}.$$

La aplicación pos es una biyección; su inversa se denota $\mathrm{assign}(k)$. El orden inducido por pos es el orden lexicográfico con x_1 como bit más significativo. Convenimos $1 =$ verdadero y $0 =$ falso. Para $x \in \Omega_n$, $F(x) = 1$ si x satisface F y $F(x) = 0$ si la falsifica.

Una *cláusula* C es una disyunción de literales; una CNF es $F = \bigwedge_{j=1}^{m} C_j$. Se supone, sin pérdida de generalidad, que las cláusulas tautológicas se han eliminado: una tautología no prohíbe ninguna asignación. La cláusula vacía se permite y prohíbe todas las asignaciones.

Para una cláusula C, el *patrón* $p(C) \in \{0, 1, \bullet\}^n$ se define por

$$p(C)_i = \begin{cases} 0, & x_i \in C, \\ 1, & \neg x_i \in C, \\ \bullet, & \text{en otro caso.} \end{cases}$$

El *subcubo* inducido por un patrón $u \in \{0, 1, \bullet\}^n$ es

$$Q(u) = \{x \in \Omega_n : \; u_i \neq \bullet \Rightarrow x_i = u_i\}.$$

Su *soporte* es $\mathrm{supp}(u) = \{i : u_i \neq \bullet\}$, su *ancho* es $k(u) = |\mathrm{supp}(u)|$ y su *volumen* es $\mathrm{vol}(u) = |Q(u)| = 2^{n-k(u)}$.

La *región prohibida* de F es

$$U(F) = \bigcup_{C \in F} Q(p(C)).$$

Entonces

$$\#\mathsf{SAT}(F) = 2^n - |U(F)|.$$

Para un entero $y \ge 0$, $\mathrm{popcnt}(y)$ denota el número de unos en su expansión binaria.

2.2. La Ecuación SAT generalizada

Para una cláusula C, sea $P_C = \mathrm{supp}(p(C))$ y $M_C = [n] \setminus P_C$. La *contribución base* es

$$B(C) = \sum_{i \in P_C} p(C)_i \, 2^{n-i}.$$

La *máscara de cláusula* es el entero

$$S_C = \bigvee_{M' \subseteq M_C} 2^{B(C) + \sum_{i \in M'} 2^{n-i}},$$

donde $\vee$ es el OR bit a bit. Equivalentemente,

$$S_C = 2^{B(C)} \prod_{i \in M_C} \left(1 + 2^{2^{n-i}}\right), \tag{1}$$

donde el producto es aritmético y no produce acarreos porque los exponentes $\sum_{i \in M'} 2^{n-i}$ son distintos para subconjuntos distintos $M' \subseteq M_C$.

El *entero de la Ecuación SAT* de F es

$$S(F) = \bigvee_{C \in F} S_C. \tag{2}$$

Escribimos $S(F)$ con exactamente 2^n bits (posiciones $0, \ldots, 2^n - 1$). El contenido de la Ecuación SAT es que el bit $\mathrm{pos}(x)$ de $S(F)$ vale 1 si y solo si $x \notin \mathsf{SAT}(F)$.

2.3. DSOP y DF-CNF

Una familia $\mathcal{U} \subseteq \{0, 1, \bullet\}^n$ es *disjunta* si $Q(u) \cap Q(v) = \varnothing$ para $u \neq v$. Una *DSOP* de $S \subseteq \Omega_n$ es una familia disjunta $\mathcal{U}$ con $\mathrm{Cov}(\mathcal{U}) = S$, donde $\mathrm{Cov}(\mathcal{U}) = \bigcup_{u \in \mathcal{U}} Q(u)$. El *tamaño mínimo de una cobertura disjunta* es

$$\chi_{\sqcup}(S) = \min\{|\mathcal{U}| : \mathcal{U} \text{ disjunta y } \mathrm{Cov}(\mathcal{U}) = S\}.$$

Una *DF-CNF* es una CNF $\mathcal{G} = \bigwedge_{j=1}^{K} D_j$ cuyas cláusulas no son tautológicas y cuyos cubos falsificadores $Q(p(D_j))$ son dos a dos disjuntos.[2]

2.4. Consultas sobre el bitvector

Fijamos la CNF F y su bitvector $S = S(F)$ de longitud 2^n, con $S[k] = \mathrm{bit}(k)$. Definimos:

$$\mathrm{bit}(k) = S[k],$$

$$\mathrm{rank}_1(k) = \sum_{j=0}^{k-1} S[j], \qquad \mathrm{rank}_0(k) = k - \mathrm{rank}_1(k),$$

$$\mathrm{select}_1(r) = \min\{k : \mathrm{rank}_1(k+1) \geq r\}, \qquad \mathrm{select}_0(r) = \min\{k : \mathrm{rank}_0(k+1) \geq r\},$$

con $0 \leq k \leq 2^n$ y $1 \leq r \leq \mathrm{rank}_1(2^n)$ (respectivamente, $r \leq \mathrm{rank}_0(2^n)$). Además,

$$\#\mathsf{SAT}(F) = \mathrm{rank}_0(2^n) = 2^n - \mathrm{popcnt}(S).$$

Un *muestreo uniforme* devuelve $\mathrm{assign}(k)$ con $k = \mathrm{select}_0(R)$, donde R es uniforme en $\{1, \ldots, \mathrm{rank}_0(2^n)\}$ (o el análogo con unos).

[2] La sigla DF alude a *falsificadores disjuntos*. No debe confundirse con la noción de forma disjunta de Darwiche–Marquis, que impone una condición de soportes disjuntos entre las partes de una conjunción; aquí la disyunción es entre los cubos falsificadores. En la literatura de compilación de conocimiento, la forma dual se denomina *DNF disjunta* o *DNF particionada* [9]; usamos DF-CNF para subrayar que la entrada es una CNF y que el objeto es la forma dual de una DSOP.

Observación 2.1 (Convención)**.** La mitad balanceada de [1] usa la convención 0 = verdadero, 1 = falso; la generalización de [2] y este artículo usan la convención estándar 1 = verdadero. Las dos codificaciones son complementarias en posición; la nota [4] detalla la equivalencia. Aquí el bit 1 marca siempre una asignación que falsifica F.

2.5. Compiladores, certificados y uniformidad

Para hablar de colapso sin ambigüedades conviene fijar qué significa *producir* una representación. Un *lenguaje de representación* $\mathcal{R}$ es un conjunto de descripciones finitas: listas de patrones para DSOP, listas de piezas afines para ADSOP, diagramas para ZDD, listas de coeficientes para Fourier, o programas para un contador. Un *compilador* para $\mathcal{R}$ es una máquina de Turing determinista M que, sobre una CNF F con n variables y m cláusulas, se detiene con una descripción $w \in \mathcal{R}$ tal que, cuando w es una lista de patrones, $\mathrm{Cov}(w) = U(F)$; en el caso de un contador, w es un programa que devuelve $\#\mathsf{SAT}(F)$. El compilador es *uniforme y polinómico* si su tiempo de ejecución y la longitud de w están acotados por $\mathrm{poly}(n, m)$.

Un *certificado* para F en $\mathcal{R}$ es una cadena w junto con una verificación de que w describe $U(F)$; el verificador es un algoritmo polinómico con acceso a F y w. La existencia de un certificado de tamaño polinómico es una afirmación no uniforme: dice que la representación compacta existe, no que se pueda encontrar. La diferencia es la misma que hay entre NP y P, y es la razón por la que el Teorema 7.1 exige un compilador y no solo un certificado. En lo que sigue, "ruta" significa una de estas representaciones junto con la pregunta por su compilador uniforme.

Definición 2.2 (Isla de tratabilidad)**.** Una familia $\mathcal{F}$ de CNF es *tratable por* $\mathcal{R}$ si existe un algoritmo uniforme que, para toda $F \in \mathcal{F}$ con n variables y m cláusulas, corre en tiempo $\mathrm{poly}(n, m)$ y produce una descripción $w \in \mathcal{R}$ de $U(F)$ de tamaño $\mathrm{poly}(n, m)$; la familia es *tratable por conteo* si existe un contador exacto que corre en tiempo $\mathrm{poly}(n, m)$ para toda $F \in \mathcal{F}$.

La Definición 2.2 es la unidad del programa de la Sección 7: en lugar de preguntar si *todas* las CNF caen en una representación, se pregunta qué familias caen y si su unión cubre el espacio. La respuesta para la DSOP ya es negativa (Proposición 4.8); para las demás representaciones es el problema abierto central.

2.6. Modelo de complejidad

Se usa el modelo de RAM con palabras de $\Theta(\log n)$ bits. Los enteros de 2^n bits no se materializan en los algoritmos comprimidos. Los contadores de rank y select tienen $O(n)$ bits; para aislar la combinatoria, contamos cada operación aritmética sobre ellos como unitaria. En el modelo de bits, cada una cuesta $O(n/\log n)$ operaciones de palabra, de modo que las cotas de tiempo se multiplican por ese factor. La evaluación de una CNF se cuenta como $O(m)$ evaluaciones de cláusula, o $O(nm)$ operaciones de bits. Los parámetros son n (variables), m (cláusulas) y K (número de cubos de la DSOP). Se distingue entre *preprocesamiento* y *consulta*: las cotas de las Secciones 5 y 6 se refieren a la consulta una vez construida la estructura o disponible el contador. La única hipótesis no demostrada en este artículo es H-ETH, que se enuncia explícitamente en el Apéndice A; dos manuscritos recientes la anuncian, pero no se usan aquí como establecidos.

3. Consultas puntuales

Proposición 3.1 (El bit es evaluación directa)**.** *Para todo* $k \in \{0, \dots, 2^n - 1\}$,

$$\mathrm{bit}(k) = 1 - F(\mathrm{assign}(k)).$$

En particular, $\mathrm{bit}(k)$ *se calcula con* $O(m)$ *evaluaciones de cláusula, o* $O(nm)$ *operaciones de bits, evaluando las* m *cláusulas de* F *en* $\mathrm{assign}(k)$.

Demostración. Por la definición de $S(F)$, el bit $\mathrm{pos}(x)$ vale 1 si y solo si x falsifica F. Tomando $x = \mathrm{assign}(k)$ se obtiene la identidad. La evaluación de una CNF con m cláusulas y n variables cuesta $O(nm)$ en el modelo de bits, y $O(m)$ si se cuenta cada cláusula como una operación sobre un patrón de n bits; usamos la cota $O(m)$ para el número de evaluaciones de cláusula. □

Observación 3.2 (Acceso puntual y acceso agregado)**.** Para π, un algoritmo tipo BBP calcula un dígito lejano sin calcular los anteriores porque π es una constante analítica con una serie rápida. Para $S(F)$, el bit $\mathrm{pos}(x)$ es la evaluación de F en x: no hay una incógnita analítica que descubrir. La dificultad no está en el acceso puntual, sino en el acceso agregado: rank, select, #SAT y muestreo.

Proposición 3.3 (Consulta de bloque)**.** *Un bloque de 2^r bits consecutivos de $S(F)$ se calcula con $O(2^r m)$ evaluaciones de cláusula evaluando F en las 2^r asignaciones correspondientes. Si se dispone de una DSOP de tamaño K, el mismo bloque se calcula en $O(2^r nK)$ operaciones sin evaluar F.*

Demostración. Inmediato de la Proposición 3.1 y de la pertenencia a un cubo. □

La Proposición 3.3 muestra que una consulta de bloque suficientemente grande puede ser más barata con la DSOP que con la CNF, pero no resuelve el problema de contar un prefijo de longitud 2^n. Ese es el objeto de las Secciones 5 y 6.

4. La forma generalizada y las DF-CNF

4.1. La Ecuación SAT para CNF arbitrarias

Teorema 4.1 (Ecuación SAT generalizada; [2])**.** *Sea F una CNF sobre n variables y sea $S(F)$ el entero definido en* (2)*. Para toda $x \in \Omega_n$, el bit* $\mathrm{pos}(x)$ *de $S(F)$ vale* 1 *si y solo si x falsifica F. En particular, $S(F)$ codifica exactamente la tabla de verdad de F y*

$$\mathrm{popcnt}(S(F)) = |U(F)|, \qquad \#\mathsf{SAT}(F) = 2^n - \mathrm{popcnt}(S(F)).$$

Demostración. Para una cláusula C, la igualdad (1) expande el producto como la suma de $2^{|M_C|}$ potencias de dos con exponentes

$$B(C) + \sum_{i \in M'} 2^{n-i}, \qquad M' \subseteq M_C.$$

Los exponentes son distintos: si dos subconjuntos M', M'' produjeran el mismo exponente, la igualdad de las sumas de potencias de dos 2^{n-i} obligaría a $M' = M''$, por la unicidad de la expansión binaria. Por tanto, la expansión binaria de S_C tiene un 1 exactamente en las posiciones de $Q(p(C))$, es decir, en las asignaciones que falsifican C. El OR (2) es la unión de estos conjuntos de posiciones, y el OR bit a bit es idempotente, de modo que no hay acarreos. Así, el bit $\mathrm{pos}(x)$ de $S(F)$ es 1 si y solo si x pertenece a algún $Q(p(C))$, esto es, si y solo si x falsifica alguna cláusula de F. Las identidades de conteo se siguen de que el peso de Hamming cuenta las posiciones con 1. □

Observación 4.2 (El caso balanceado)**.** Si toda cláusula tiene ancho n, entonces $M_C = \varnothing$ y $S_C = 2^{B(C)}$; la suma aritmética coincide con el OR cuando las cláusulas no comparten asignación falsificante. La forma generalizada sustituye la suma por OR para absorber solapamientos y permite anchos arbitrarios. La forma balanceada es, por tanto, el caso particular sin variables libres.

4.2. DF-CNF: la dualidad exacta

Teorema 4.3 (DSOP $\leftrightarrow$ DF-CNF)**.** *Para toda CNF F sobre n variables,*

$$\chi_{\sqcup}(U(F)) = \min\{K : \exists \mathcal{G} \text{ DF-CNF con } K \text{ cláusulas y } \mathcal{G} \equiv F\}.$$

Además, si $\mathcal{U} = \{u_1, \dots, u_K\}$ es una cobertura disjunta de $U(F)$, la CNF

$$\mathcal{G}_{\mathcal{U}} = \bigwedge_{j=1}^{K} C_{u_j}, \qquad C_u = \Big(\bigvee_{i:u_i=0} x_i \Big) \vee \Big(\bigvee_{i:u_i=1} \neg x_i \Big),$$

es una DF-CNF equivalente a F con $Q(p(C_{u_j})) = Q(u_j)$.

Demostración. Para un patrón u, la cláusula C_u contiene exactamente los literales que se vuelven falsos en u: si $u_i = 0$, el literal x_i es falso cuando $x_i = 0$; si $u_i = 1$, el literal $\neg x_i$ es falso cuando $x_i = 1$; si $u_i = \bullet$, no hay literal. Por tanto, $Q(p(C_u)) = Q(u)$.

Si $\mathcal{U}$ es disjunta y cubre $U(F)$, entonces

$$\mathsf{SAT}(\mathcal{G}_{\mathcal{U}}) = \bigcap_{u \in \mathcal{U}} (\Omega_n \setminus Q(u)) = \Omega_n \setminus \bigcup_{u \in \mathcal{U}} Q(u) = \Omega_n \setminus U(F) = \mathsf{SAT}(F).$$

Luego $\mathcal{G}_{\mathcal{U}} \equiv F$ y sus cubos falsificadores son los $Q(u)$, disjuntos. Recíprocamente, si $\mathcal{G} = \bigwedge D_j$ es una DF-CNF equivalente a F, entonces $U(\mathcal{G}) = U(F)$ y los $Q(p(D_j))$ forman una cobertura disjunta de $U(F)$ de tamaño K. Por tanto el mínimo sobre DF-CNFs es al menos $\chi_{\sqcup}(U(F))$; la construcción anterior da la otra desigualdad. Los casos extremos $U(F) = \varnothing$ y $U(F) = \Omega_n$ se cubren con $K = 0$ (CNF vacía) y $K = 1$ (cláusula vacía), respectivamente. □

Corolario 4.4 (Acarreo cero y COVERTRACE)**.** *Sea $\mathcal{U}$ una cobertura disjunta de $U(F)$ de tamaño K. Entonces*

$$S(F) = \sum_{u \in \mathcal{U}} S_{C_u}$$

como suma aritmética, sin acarreos, y el número de cláusulas de la DF-CNF $\mathcal{G}_{\mathcal{U}}$ es K. En particular, la salida de COVERTRACE [3] es una DF-CNF equivalente a F.

Demostración. Los soportes de las máscaras S_{C_u} son los cubos disjuntos $Q(u)$; por tanto, la suma aritmética y el OR coinciden. La segunda parte es el Teorema 4.3. □

Ejemplo 4.5 (Cláusula ancha)**.** Sea C una cláusula de ancho w. Su región prohibida es un único cubo de volumen 2^{n-w}; la DSOP tiene $K = 1$ cubo y la DF-CNF tiene una cláusula. La forma balanceada requeriría 2^{n-w} singletons para representar el mismo cubo. Esta es la razón por la que la forma generalizada es la adecuada para rank y select.

Ejemplo 4.6 (Paridad)**.** Sea $O_n = \{x : \bigoplus_i x_i = 1\}$ y sea F_n la CNF con una cláusula C_a por cada $a \in O_n$, construida de modo que $Q(p(C_a)) = \{a\}$. Entonces $U(F_n) = O_n$, $\chi_{\sqcup}(O_n) = 2^{n-1}$ y $S(F_n)$ es el entero de Thue–Morse de [4], con $\mathrm{popcnt}(S(F_n)) = 2^{n-1}$. La cota es ajustada: cualquier cubo con al menos una coordenada libre contiene asignaciones de ambas paridades, de modo que una cobertura disjunta de O_n usa solo singletons; hay 2^{n-1}. La CNF F_n tiene 2^{n-1} cláusulas y, por tanto, este ejemplo no separa CNF de tamaño polinómico de DSOP de tamaño polinómico: la entrada crece con la representación. La separación real la da la construcción de Sasao, que se presenta a continuación.

Lema 4.7 (Sasao; recogido por Bernasconi et al.)**.** *Existe una sucesión de funciones booleanas G_n sobre $N = O(n)$ variables, definida para n par, con una SOP de $n/2$ términos y una DSOP mínima de $2^{n/2} - 1$ cubos.*

Demostración. La construcción es la de Sasao [25], pp. 8:11–8:20. Bernasconi, Ciriani, Luccio y Pagli [26], Introducción, p. 2 de la versión arXiv:1204.5306, la recogen y la presentan como un caso en el que el crecimiento exponencial aparece también para soluciones mínimas. No reproducimos aquí la prueba de minimalidad; el lema se usa como cota inferior citada. □

Proposición 4.8 (La DSOP no es universal)**.** *Existe una familia $\{F_n\}$ de CNF con $N = O(n)$ variables y $O(n)$ cláusulas tal que*

$$\chi_{\sqcup}(U(F_n)) = 2^{\Omega(n)}.$$

Demostración. Por el Lema 4.7, sea $G_n = T_1 \vee \cdots \vee T_{n/2}$ sobre $N = O(n)$ variables, donde cada T_i es una conjunción de literales, y sea

$$F_n = \bigwedge_{i=1}^{n/2} \neg T_i.$$

Cada $\neg T_i$ es una cláusula, de modo que F_n tiene $n/2$ cláusulas y $N = O(n)$ variables. Además, $F_n(x) = 0$ si y solo si algún $T_i(x) = 1$, es decir, $U(F_n) = \{x : G_n(x) = 1\}$. Por tanto, una cobertura disjunta de $U(F_n)$ es exactamente una DSOP de G_n, y $\chi_{\sqcup}(U(F_n)) = 2^{n/2} - 1 = 2^{\Omega(n)}$. □

Observación 4.9 (Qué dice y qué no dice la separación)**.** La Proposición 4.8 es una cota inferior incondicional para el lenguaje de cubos disjuntos. No es una cota inferior para SAT ni para #SAT: la CNF F_n puede seguir siendo fácil para otra representación, por ejemplo una ADSOP, una ZDD o un contador especializado. Lo que descarta es la esperanza de que un compilador DSOP uniforme y polinómico cubra todas las CNF; esa ruta, por sí sola, no puede dar P = NP. El programa de la Sección 7 consiste en determinar qué representación, si alguna, ocupa el lugar que la DSOP deja vacío.

5. Rank y select desde una DSOP

5.1. Conteo de prefijos en un cubo

Para un patrón u y un entero $0 \le k \le 2^n$, sea

$$\mathrm{pc}(u, k) = \big|Q(u) \cap \{x : \mathrm{pos}(x) < k\}\big|.$$

Escribimos k en binario con n bits, $k = (k_1, \ldots, k_n)$, y definimos el número de coordenadas libres de u en las posiciones $i, \ldots, n$ como

$$f_u(i) = \big|\{j \ge i : u_j = \bullet\}\big|.$$

Convenimos $f_u(n+1) = 0$.

Lema 5.1 (Prefijo de un cubo)**.** *Para todo patrón u y todo $0 \le k \le 2^n$, $\mathrm{pc}(u, k)$ se calcula en $O(n)$ operaciones aritméticas y $O(n)$ bits adicionales. Para $k = 2^n$, $\mathrm{pc}(u, 2^n) = \mathrm{vol}(u)$.*

Demostración. Para $k = 2^n$ la identidad es inmediata. Para $0 \le k < 2^n$, escribimos $k = (k_1, \ldots, k_n)$ y definimos C_i como el número de asignaciones de $Q(u)$ que coinciden con k en las primeras $i-1$ posiciones y son estrictamente menores en la posición i o en alguna posterior, con $C_{n+1} = 0$. La recurrencia es

$$C_i = \begin{cases} 2^{f_u(i+1)}, & u_i \neq \bullet \text{ y } u_i < k_i, \\ 0, & u_i \neq \bullet \text{ y } u_i > k_i, \\ C_{i+1}, & u_i \neq \bullet \text{ y } u_i = k_i, \\ C_{i+1}, & u_i = \bullet \text{ y } k_i = 0, \\ 2^{f_u(i+1)} + C_{i+1}, & u_i = \bullet \text{ y } k_i = 1. \end{cases}$$

La recurrencia recorre exactamente las opciones del bit i: las que ya hacen el prefijo menor contribuyen con todas sus extensiones, $2^{f_u(i+1)}$; la que coincide con k_i continúa; las que son mayores no contribuyen. Por inducción descendente sobre i, C_i cuenta las asignaciones de $Q(u)$ que coinciden con k en las primeras $i-1$ posiciones y son menores en la posición i o en alguna posterior, y C_1 es $\mathrm{pc}(u,k)$. El recorrido tiene n pasos; el contador y las potencias $2^{f_u(i)}$ ocupan $O(n)$ bits. □

Observación 5.2. El procedimiento anterior es un autómata de dos estados sobre los bits de k; la bandera *tight* es el único estado necesario. No se materializa el cubo.

5.2. El motor DSOP

Teorema 5.3 (Rank/select con una DSOP)**.** *Sea $\mathcal{U}$ una cobertura disjunta de $U(F)$ de tamaño K. Entonces:*

(a) $\mathrm{rank}_1(k) = \sum_{u\in\mathcal{U}} \mathrm{pc}(u,k)$ *y* $\mathrm{rank}_0(k) = k - \mathrm{rank}_1(k)$ *se calculan en $O(nK)$ operaciones aritméticas.*

(b) $\#\mathsf{SAT}(F) = 2^n - \sum_{u\in\mathcal{U}} \mathrm{vol}(u)$ *se calcula en $O(K)$ operaciones aritméticas.*

(c) $\mathrm{select}_0(r)$ *y* $\mathrm{select}_1(r)$ *se calculan en $O(n^2K)$ operaciones aritméticas usando búsqueda binaria sobre* rank_0 *o* rank_1*.*

(d) Si $\mathrm{rank}_0(2^n) > 0$*, un cero uniforme se obtiene en $O(n^2K)$ operaciones aritméticas eligiendo R uniforme en $\{1,\dots,\mathrm{rank}_0(2^n)\}$ y devolviendo* $\mathrm{select}_0(R)$*. El análogo vale para unos.*

Demostración. (a) La disyunción de $\mathcal{U}$ implica

$$\{x : \mathrm{pos}(x) < k\} \cap U(F) = \bigsqcup_{u\in\mathcal{U}} \big(\{x : \mathrm{pos}(x) < k\} \cap Q(u)\big),$$

y los cardinales se suman. Cada término es $\mathrm{pc}(u,k)$, calculable en $O(n)$ por el Lema 5.1. La identidad para rank_0 es inmediata.

(b) $\#\mathsf{SAT}(F) = 2^n - |U(F)|$ y $|U(F)| = \sum_{u\in\mathcal{U}} \mathrm{vol}(u)$ por disyunción.

(c) La función $k \mapsto \mathrm{rank}_0(k)$ es no decreciente y $\mathrm{rank}_0(2^n) = \#\mathsf{SAT}(F)$. La búsqueda binaria sobre $k \in \{0,\dots,2^n-1\}$ usa $O(n)$ consultas de rank_0; cada consulta cuesta $O(nK)$ por (a), luego $O(n^2K)$. El mismo argumento vale para select_1 con rank_1.

(d) Si R es uniforme en $\{1,\dots,\mathrm{rank}_0(2^n)\}$, la aplicación $R \mapsto \mathrm{select}_0(R)$ es una biyección entre ese conjunto y las posiciones de los ceros, porque select_0 es estrictamente creciente. Por tanto, el cero devuelto es uniforme. □

Corolario 5.4 (Cota polinómica, dado el certificado)**.** *Sea $\mathcal{U}$ una cobertura disjunta de $U(F)$ de tamaño $K = \mathrm{poly}(n,m)$. Entonces, dado $\mathcal{U}$, las consultas* bit, $\#\mathsf{SAT}$, rank *y* select *se responden en tiempo polinómico en n, m, K. Si además $\mathcal{U}$ se construye en tiempo polinómico a partir de F, entonces SAT y $\#\mathsf{SAT}$ para F se deciden y cuentan en tiempo polinómico; en particular, se puede extraer un testigo de satisfacibilidad o de falsificación y verificarlo con una evaluación de F.*

Demostración. La evaluación puntual es $O(m)$; las demás cotas son las del Teorema 5.3. Para un testigo, se usa $\mathrm{select}_0(1)$ o $\mathrm{select}_1(1)$ y se evalúa F en la asignación resultante. La segunda afirmación es la aplicación del Teorema 5.3 a la cobertura ya construida. □

Observación 5.5 (El coste dominante es K)**.** El Teorema 5.3 no supone que K sea pequeño. Si $K = 2^{n-1}$ (paridad), el motor es exponencial en n; si $K = 1$ (cláusula ancha), todas las consultas son $O(n)$ o $O(n^2)$. La pregunta práctica es si la DSOP de una familia de CNF es pequeña; esa es exactamente la pregunta de la compilación de conocimiento [9, 3]. En la paridad, la CNF de entrada también tiene 2^{n-1} cláusulas; el ejemplo muestra el límite del lenguaje de cubos, pero no

es la separación entre CNF de tamaño polinómico y DSOP de tamaño polinómico. Esa separación existe y la da otra familia: la Proposición 4.8, con $O(n)$ cláusulas y $K = 2^{\Omega(n)}$.

Ejemplo 5.6 (Cláusula ancha, cálculo explícito)**.** Sea $n = 4$ y $C = (x_1 \vee x_2)$, de ancho $w = 2$. Entonces $U(C) = \{0000, 0001, 0010, 0011\}$ y $S(C) = 1 + 2 + 4 + 8 = 15$. La DSOP tiene $K = 1$ cubo $Q(00 \bullet \bullet)$ (con $x_1 = x_2 = 0$), de volumen 4. Para $k = 3$, $\mathrm{pc}(u, 3) = 3$ porque las posiciones $0, 1, 2$ están en el cubo y la posición 3 no es menor que 3. Para $k = 4$, $\mathrm{pc}(u, 4) = 4$. La consulta es $O(n)$.

5.3. De la CNF a la DSOP

El Teorema 5.3 supone una cobertura disjunta $\mathcal{U}$. COVERTRACE [3] construye una para $U(F)$ mediante *CubeDiff* / *AddCube*; su tamaño de salida es K. El Teorema 4.3 garantiza que esa salida es una DF-CNF equivalente a F. Si K es pequeño, las consultas son polinómicas una vez construida $\mathcal{U}$. El coste de construirla es el de COVERTRACE: su cota es $O((T + m)c(n))$, donde T cuenta las piezas producidas con multiplicidad; la cota es en T, no en el tamaño final K. La paridad muestra que K puede ser 2^{n-1}; la Proposición 4.8 muestra que hay familias con $K = 2^{\Omega(n)}$ y solo $O(n)$ cláusulas. La Sección 7 analiza el alcance de esta cota y su relación con el tamaño de la entrada.

6. Un oráculo de conteo para rank y select

6.1. El comparador de prefijos

Lema 6.1 (CNF comparadora)**.** *Para cada n y cada $0 \le k \le 2^n$ existe una CNF $\mathcal{C}_k$ con $O(n)$ variables y $O(n)$ cláusulas, sobre un conjunto de variables que incluye a $x_1, \ldots, x_n$, tal que la codificación Tseitin del comparador tiene una única extensión para cada $x \in \Omega_n$, y $\mathcal{C}_k$ (con la salida forzada a 1) es satisfacible si y solo si $\mathrm{pos}(x) < k$. Para $k = 2^n$, la condición es vacía.*

Demostración. Se construye un comparador estándar de dos números de n bits con un autómata de tres estados (menor, igual, mayor) que recorre las posiciones de izquierda a derecha. Cada transición es una compuerta booleana de abanico acotado, y el circuito tiene $O(n)$ compuertas. La codificación Tseitin de cada compuerta con una variable auxiliar produce $O(n)$ cláusulas y fuerza un valor único para cada auxiliar dada x. Se añade la cláusula unitaria que fija la salida "menor" a 1. Por tanto, cada x tiene a lo sumo una extensión satisfactoria, y esa extensión existe exactamente cuando la salida del comparador es verdadera, esto es, cuando $\mathrm{pos}(x) < k$. Para $k = 2^n$ se usa la CNF vacía, pues todo x cumple la condición. □

6.2. Reducción de rank a conteo exacto

Teorema 6.2 (Rank desde un contador exacto)**.** *Sea A un algoritmo exacto que, dada una CNF con N variables y M cláusulas, devuelve su número de modelos en tiempo $T(N, M)$ y espacio $S(N, M)$. Entonces, para toda CNF F sobre n variables y todo prefijo $0 \le t \le 2^n$,*

$$\mathrm{rank}_0(t) = \#\mathsf{SAT}\big(F \wedge \mathcal{C}_t\big),$$

y $\mathrm{rank}_0(t)$ se calcula en tiempo $T(n + O(n), m + O(n)) + O(n)$ y espacio $S(n + O(n), m + O(n)) + O(n)$. Además, $\mathrm{select}_0(r)$ requiere $O(n)$ llamadas a A y se calcula en tiempo $O\big(n \cdot (T(n + O(n), m + O(n)) + n)\big)$; el muestreo uniforme requiere $O(n)$ llamadas en total.

Demostración. Por el Lema 6.1, las extensiones auxiliares de $\mathcal{C}_t$ son únicas para cada x. Por tanto, el número de modelos de $F \wedge \mathcal{C}_t$ sobre todas las variables es exactamente el número de

$x \in \mathsf{SAT}(F)$ con $\mathrm{pos}(x) < t$, que es $\mathrm{rank}_0(t)$ por definición. La cota de tiempo es la de A sobre la CNF combinada, más el coste de construir el comparador. Para select_0, se hace búsqueda binaria sobre t como en el Teorema 5.3: $O(n)$ consultas de rank_0; la corrección es la monotonía de rank_0. Para el muestreo, se consulta $\mathrm{rank}_0(2^n)$, se elige R uniforme en $\{1, \ldots, \mathrm{rank}_0(2^n)\}$ y se devuelve $\mathrm{select}_0(R)$. □

Observación 6.3. El Teorema 6.2 es válido para cualquier contador exacto: no depende de qué algoritmo se use. Su interés práctico depende de que A sea más rápido que materializar $S(F)$ y de que la CNF combinada conserve una estructura favorable, por ejemplo que siga siendo k-CNF o que su DSOP no explote.

6.3. La instanciación H-ETH

La instanciación subexponencial con H-ETH se presenta en el Apéndice A. Es una nota condicional y puede omitirse en una primera lectura: no interviene en los resultados de las Secciones 4–5 ni en el mapa de barreras.

7. La frontera de complejidad: mapa de barreras y programa hacia $\mathsf{P} = \mathsf{NP}$

El motor de las Secciones 5–6, junto con las optimizaciones de la Sección 9, responde consultas sobre $S(F)$ sin materializar sus 2^n bits. Como $\mathrm{rank}_0(2^n) = \#\mathsf{SAT}(F)$, una lectura apresurada podría concluir que el motor decide SAT en tiempo polinómico. La conclusión no se sigue. La razón es una separación de costes: las cotas de consulta son polinómicas en n y en el tamaño K del certificado, pero el certificado y el contador no son gratuitos. Esta sección precisa la separación, recorre las rutas por las que un colapso podría intentarse y convierte el resultado negativo en un programa de investigación. La pregunta que organiza el programa no es "¿resuelve el motor $\mathsf{P} = \mathsf{NP}$?" sino "¿qué tendría que demostrarse para que una de estas rutas lo resolviera, y qué se sabe ya sobre cada una?".

7.1. La condición de colapso

Para decidir SAT basta con calcular $\#\mathsf{SAT}(F)$ y comparar con cero. Por tanto, cualquier algoritmo que calcule $\#\mathsf{SAT}$ para toda CNF en tiempo polinómico implica $\mathsf{P} = \mathsf{NP}$; de hecho, implica $\#\mathsf{P} \subseteq \mathsf{FP}$ y, por el teorema de Toda [15], $\mathsf{PH} = \mathsf{P}$.

Teorema 7.1 (Colapso bajo compilación uniforme)**.** *Si existe un algoritmo determinista que, para toda CNF F sobre n variables y m cláusulas, se ejecuta en tiempo $\mathrm{poly}(n, m)$ y produce una familia disjunta $\mathcal{U}$ con $\mathrm{Cov}(\mathcal{U}) = U(F)$ y $|\mathcal{U}| \leq \mathrm{poly}(n, m)$, entonces $\#\mathsf{SAT} \in \mathsf{FP}$ y $\mathsf{PH} = \mathsf{P}$. En particular, $\mathsf{P} = \mathsf{NP}$.*

Demostración. Dada F, el algoritmo produce $\mathcal{U}$ en tiempo polinómico. Por disyunción,

$$|U(F)| = \sum_{u \in \mathcal{U}} 2^{n-k(u)},$$

que se calcula en tiempo polinómico. Por tanto, $\#\mathsf{SAT}(F) = 2^n - |U(F)| \in \mathsf{FP}$. Como $\#\mathsf{SAT}$ es $\#\mathsf{P}$-completo [14], $\#\mathsf{P} \subseteq \mathsf{FP}$. Por Toda [15], $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}} = \mathsf{P}$; como $\mathsf{P} \subseteq \mathsf{PH}$, se obtiene $\mathsf{PH} = \mathsf{P}$ y, en particular, $\mathsf{P} = \mathsf{NP}$. □

Corolario 7.2 (La barrera en forma contrapositiva)**.** *Si $\mathsf{PH} \neq \mathsf{P}$, entonces no existe un compilador uniforme polinómico de DSOP con salida polinómica para toda CNF. El mismo enunciado vale para ADSOP, ZDD, espectro de Fourier y para un contador exacto polinómico.*

Demostración. Es el contrapositivo del Teorema 7.1 y de las variantes que se obtienen al sustituir la representación por cualquiera de las otras; en todos los casos, la representación de tamaño polinómico permite calcular #SAT en tiempo polinómico, y el argumento de Toda da PH = P. □

El teorema es la misma implicación que el Teorema 7.4 de [3]. Su hipótesis es fuerte: un compilador uniforme, polinómico y con salida polinómica para toda CNF. El motor de este artículo no es ese compilador. Toma una cobertura $\mathcal{U}$ como entrada; la corrección y el coste de las consultas se demuestran para $\mathcal{U}$ ya construida. La pregunta es, entonces, si alguna de las rutas del motor proporciona la hipótesis del Teorema 7.1. En este artículo la respuesta es no para la DSOP, y queda abierta para las demás: ninguna de las rutas se demuestra uniforme y polinómica para toda CNF, y cada una falla por una razón distinta.

7.2. La rama DSOP: el certificado y su tamaño

El Teorema 5.3 es polinómico en n y K. Si K es polinómico en el tamaño de la entrada $n+m$, las consultas son polinómicas en la entrada; si K es exponencial en n pero la entrada también lo es, el coste puede ser exponencial en n y polinómico en m. La paridad ilustra el segundo caso: $\chi_{\sqcup}(O_n) = 2^{n-1}$ (Ejemplo 4.6), pero la CNF de paridad tiene 2^{n-1} cláusulas, de modo que $K = 2^{n-1}$ es lineal en m. Ese ejemplo, por sí solo, no separa CNF de tamaño polinómico de DSOP de tamaño polinómico. La separación la da la construcción de Sasao y Bernasconi et al.: la Proposición 4.8 exhibe una familia con $m = O(n)$ y $\chi_{\sqcup} = 2^{\Omega(n)}$. Por tanto, *no existe* un compilador DSOP uniforme polinómico para toda CNF: la salida no puede ser polinómica en la entrada para esa familia. La ruta DSOP queda cerrada como ruta universal; no queda cerrada como isla de tratabilidad. La equivalencia DSOP–DF-CNF del Teorema 4.3 no elimina esta dificultad: es una reescritura exacta de la misma región prohibida, y la DF-CNF resultante tiene exactamente K cláusulas; el parámetro K sigue siendo el certificado.

La hipótesis del Teorema 7.1 es un compilador uniforme polinómico en el tamaño de la entrada. El artículo no lo proporciona. COVERTRACE tiene una cota $O((T+m)c(n))$, donde T cuenta las piezas producidas con multiplicidad; es una cota parametrizada, no una garantía de tiempo polinómico en $n+m$ cuando K crece. El tope $K_{\max}$ limita el tamaño de la salida; no convierte a COVERTRACE en un compilador polinómico en el tamaño de la entrada. Para obtener el colapso haría falta un compilador uniforme polinómico, y el artículo no lo proporciona.

La consecuencia correcta es local: si una familia de CNF admite DSOP de tamaño polinómico en el tamaño de la entrada y un compilador polinómico, entonces SAT y #SAT son polinómicos en esa familia. Es un resultado estructural, del mismo tipo que la tratabilidad de las fórmulas de ancho de árbol acotado, no un colapso de clases.

7.3. La rama de conteo: #SAT y la dirección de la reducción

El Teorema 6.2 demuestra

$$\mathrm{rank}_0(t) = \#\mathsf{SAT}(F \wedge \mathcal{C}_t).$$

Es una reducción polinómica de rank a #SAT: muestra que rank no es más difícil que #SAT. No muestra que #SAT sea fácil. La dirección importa. En el otro sentido, $\#\mathsf{SAT}(F) = \mathrm{rank}_0(2^n)$ es una sola consulta de rank; por tanto, rank y #SAT son equivalentes en tiempo polinómico. El oráculo de rank es exactamente tan difícil como #SAT.

El comparador de prefijos es parsimonioso: cada x tiene una única extensión a las variables auxiliares, de modo que el conteo se preserva. La codificación añade $O(n)$ variables y $O(n)$ cláusulas, pero no simplifica F; si F es difícil para #SAT, $F \wedge \mathcal{C}_t$ también lo es. La reducción sirve para calcular rank a partir de un contador, no para construir el contador.

Si el contador A fuera polinómico para toda CNF, se seguiría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$. Pero A es un oráculo; el Teorema 6.2 es válido para cualquier contador exacto y no afirma que exista uno polinómico. La hipótesis H-ETH da un contador subexponencial, no polinómico. Esa diferencia es decisiva y se analiza a continuación.

7.4. La rama H-ETH: subexponencial no es polinómico

La hipótesis H-ETH se enuncia en el Apéndice A; aquí solo se usa su consecuencia. H-ETH proporciona un contador en tiempo $2^{o(n)}\,\mathrm{poly}(n, m)$, no en tiempo $\mathrm{poly}(n, m)$. La distancia entre ambos regímenes es el objeto de P frente a NP. Un algoritmo subexponencial para $\#\mathsf{SAT}$ refuta $\#\mathsf{ETH}$ y, por tanto, ETH, pero no implica $\mathsf{P} = \mathsf{NP}$.

La relación lógica es precisa. Si $\mathsf{P} = \mathsf{NP}$, entonces $3\mathsf{SAT}$ tendría un algoritmo polinómico y, por tanto, uno de tiempo $2^{o(n)}$, lo que contradice ETH. Así, $\mathsf{P} = \mathsf{NP}$ implica que ETH es falsa; equivalentemente, ETH implica $\mathsf{P} \neq \mathsf{NP}$. La negación de ETH es una condición necesaria para $\mathsf{P} = \mathsf{NP}$, no una condición suficiente. H-ETH, aun si se verificara, dejaría abierta la pregunta de si el contador puede bajarse a tiempo polinómico. El Corolario A.3 es un resultado subexponencial condicional, no una prueba de $\mathsf{P} = \mathsf{NP}$.

Para CNF de ancho arbitrario hay una segunda limitación: la codificación a k-CNF añade variables auxiliares y la cota resultante es en el número de variables codificadas, no necesariamente en las n originales. La ruta DSOP no tiene esa restricción de ancho; de ahí la complementariedad de las dos rutas.

7.5. La rama de Fourier

Si el espectro de Walsh–Hadamard de $S(F)$ tiene s coeficientes no nulos y se conocen, el motor de Fourier de la Sección 9.1 responde rank en $O(sn)$ y select en $O(sn^2)$, y $\#\mathsf{SAT}$ es $2^n - \hat{f}(\varnothing)$. La paridad tiene $s = 2$, de modo que Fourier comprime exactamente lo que la DSOP no comprime. Pero el espectro puede ser denso: una cláusula ancha de ancho w tiene $s = 2^w$, y una función aleatoria tiene un espectro denso. El algoritmo de [7] calcula la transformada densa de longitud 2^n en $O(2^n n^{1-\delta})$, todavía exponencial en n; no es un compilador de espectro disperso. Un espectro de tamaño polinómico construido en tiempo polinómico para toda CNF daría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$; el artículo no lo proporciona. La barrera es la misma que en las otras ramas: la representación debe ser pequeña y construible en tiempo polinómico.

7.6. La rama ZDD y la no uniformidad

Una ZDD de tamaño Z responde las consultas en $O(n)$, pero Z puede ser exponencial. Si se construye desde una DSOP de K cubos, $Z = O(nK)$; la reducción puede disminuirla, como en la paridad, donde $Z = O(n)$ y $K = 2^{n-1}$. La ventaja de la ZDD es real, pero presupone la construcción. La compilación directa de una CNF a una ZDD pequeña, sin pasar por la DSOP, es un problema abierto (Sección 13). Si existiera un algoritmo uniforme que, para toda CNF con n variables y m cláusulas, corriera en tiempo $\mathrm{poly}(n, m)$ y produjera una ZDD de tamaño $\mathrm{poly}(n, m)$, el conteo sobre la ZDD daría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por el mismo argumento, $\mathsf{PH} = \mathsf{P}$. La cota debe ser en n, m, no en el tamaño de la ZDD: un algoritmo polinómico en Z que produzca una ZDD exponencial no sirve para el colapso. El artículo no proporciona ese algoritmo; la ZDD es una optimización condicional al tamaño de la estructura.

Hay una distinción adicional entre certificado y algoritmo. Una DSOP o una ZDD de tamaño polinómico para cada F es un certificado no uniforme: dice que la representación existe, no que se pueda encontrar en tiempo polinómico. P frente a NP es una pregunta sobre algoritmos uniformes. Un certificado polinómico por instancia no basta por sí solo para responderla; hace falta un

compilador uniforme. El Teorema 7.1 formula esa condición y muestra que, si se cumpliera, el colapso sería aún más fuerte: $\mathsf{PH} = \mathsf{P}$.

7.7. La rama estocástica y la rama afín

La Sección 10 añade dos rutas que no aparecen en el motor de las secciones anteriores. La estocástica no es una ruta al colapso en el mismo sentido: el muestreo y el hashing dan error relativo, no exactitud. Un FPRAS para #SAT para toda CNF implicaría $\mathsf{NP} \subseteq \mathsf{BPP}$: una instancia con $\#\mathsf{SAT} = 0$ devuelve 0 y una con $\#\mathsf{SAT} > 0$ devuelve un valor positivo con probabilidad al menos $1-\delta$, de modo que SAT se decide en BPP. Karp–Luby da un FPRAS para $|U(F)|$, pero la resta $2^n - |U|$ no conserva el error relativo. Por tanto, la aleatoriedad cambia la garantía de exacta a aproximada; no proporciona $\#\mathsf{SAT} \in \mathsf{FP}$.

La rama afín sí es exacta. La ADSOP contiene a la paridad en una pieza y responde conteo, rango y selección en tiempo polinómico en su tamaño (Teorema 10.4). Si existiera un compilador ADSOP uniforme polinómico para toda CNF, se seguiría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$. El prototipo de la Sección 10.4 no es ese compilador: usa un oráculo SAT, hace $O(n^2)$ consultas por nodo y puede explorar un número exponencial de nodos. La cota de paridad de la DSOP no se hereda, pero la ADSOP tiene su propia barrera de compilación, que queda abierta.

7.8. Resumen de las rutas

La Tabla 1 resume las siete rutas. La columna "si fuera polinómica" indica la consecuencia de convertir la ruta en un algoritmo uniforme polinómico para toda CNF; la última columna indica lo que el artículo ofrece.

Ruta	Qué se necesita	Si fuera polinómica	Qué ofrece el artículo
DSOP	compilador uniforme que produzca $\mathcal{U}$ de tamaño $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	consultas polinómicas dado $\mathcal{U}$ (Teorema 5.3)
Contador	contador exacto polinómico para toda CNF	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	reducción rank $\leq$ #SAT (Teorema 6.2); H-ETH da $2^{o(n)}$
ZDD	construcción uniforme polinómica de una ZDD polinómica	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	optimización condicional (Sección 9)
Fourier	espectro WHT de tamaño $\mathrm{poly}(n, m)$ construido en tiempo $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	consultas $O(sn)$ dado el espectro (Sección 9.1)
ADSOP	compilador uniforme de ADSOP de tamaño $\mathrm{poly}(n, m)$	$\#\mathsf{SAT} \in \mathsf{FP}$, $\mathsf{PH} = \mathsf{P}$	consultas exactas dado ADSOP (Teorema 10.4); prototipo (Sección 10.4)
Estocástico	FPRAS de #SAT con error relativo para toda CNF	FPRAS para $\#\mathsf{SAT} \Rightarrow \mathsf{NP} \subseteq \mathsf{BPP}$ (aproximación, no exactitud)	muestreo, hashing y Karp–Luby (Sección 10.2)
Bit	evaluación de F en una asignación	ningún colapso	$O(m)$ por consulta (Proposición 3.1)

Tabla 1: Las rutas del motor y la condición exacta de colapso. La ruta DSOP está descartada como ruta universal por la Proposición 4.8; las demás siguen abiertas.

7.9. Certificados, verificación y complejidad de pruebas

La distinción entre certificado y algoritmo merece un tratamiento propio porque la DSOP se parece a un certificado de NP y, sin embargo, no lo es. Un certificado de satisfacibilidad es una asignación que se verifica en $O(m)$. Un certificado DSOP es una lista de cubos que cubre $U(F)$; verificarlo exige comprobar que la unión es exactamente $U(F)$, y la contención de un cubo en una unión de cubos es coNP-difícil en general. La razón es directa: el complemento de la contención es la satisfacibilidad de la CNF formada por los complementos de los cubos, restringida al cubo, y ese problema es NP-completo. Por eso el certificado DSOP no es un certificado de NP en el sentido habitual: la verificación no es polinómica sin una traza de construcción.

Esta observación conecta el parámetro K con la complejidad de pruebas. El tamaño de una DSOP es la medida de una refutación en un sistema de prueba natural cuando F es insatisfacible: en ese caso $U(F) = \Omega_n$, y una DSOP de Ω_n con K cubos es una refutación de tamaño K en el sistema de cubos disjuntos. Comparar K con las medidas de resolución, *cutting planes* o *polynomial calculus* es un programa en sí mismo; las cotas inferiores de Ben-Sasson y Wigderson [33] y la literatura de complejidad de pruebas [34] dan el marco. La Proposición 4.8 es una cota inferior de este tipo para una familia concreta, aunque no necesariamente insatisfacible: hay CNF con $O(n)$ cláusulas cuya región prohibida tiene una cobertura disjunta mínima de tamaño $2^{\Omega(n)}$.

Problema abierto (Verificación de cobertura)**.** Determinar si una DSOP de $U(F)$ puede verificarse en tiempo polinómico con una traza de construcción de tamaño polinómico, y caracterizar las familias para las que la verificación sin traza es polinómica. La contención de un cubo en una unión de cubos es coNP-difícil en general, de modo que el certificado debe incluir algo más que la lista de cubos.

La consecuencia para el programa es clara: un certificado de tamaño polinómico para cada F no resuelve $\mathsf{P} = \mathsf{NP}$; lo que hace falta es un *buscador* de certificados. La complejidad de pruebas estudia la existencia y el tamaño de los certificados; la compilación estudia su búsqueda. El motor de este artículo es la interfaz entre ambos: dada una representación, responde consultas; dada una traza, la verifica; lo que no hace es encontrar la representación.

7.10. Islas de tratabilidad y la pregunta de cobertura

La Definición 2.2 y la Tabla 1 permiten formular el programa sin ambigüedad. Cada representación define una isla: las CNF que admiten un compilador uniforme polinómico para esa representación. La primera observación es que todas las islas están dentro de P.

Proposición 7.3 (Las islas están en P)**.** *Sea $\mathcal{F}$ una familia de CNF tratable por DSOP, ADSOP, ZDD, espectro de Fourier o conteo, en el sentido de la Definición 2.2. Entonces SAT y $\#\mathsf{SAT}$ se resuelven en tiempo polinómico para toda $F \in \mathcal{F}$.*

Demostración. Por definición, existe un compilador uniforme polinómico que produce la representación. Para DSOP, el Teorema 5.3 da $\#\mathsf{SAT}$ en $O(K)$ y rank en $O(nK)$; para ADSOP, el Teorema 10.4 da el conteo y las consultas; para ZDD, el conteo de conjuntos es $|U(F)|$ y $\#\mathsf{SAT} = 2^n - |U(F)|$, calculable recorriendo el diagrama; para Fourier, $\#\mathsf{SAT} = 2^n - \hat{f}(\varnothing)$ y el espectro se supone construido; para conteo, el contador es el algoritmo mismo. En todos los casos, SAT se decide comparando $\#\mathsf{SAT}$ con cero. La composición de un compilador polinómico con una consulta polinómica es polinómica. □

La pregunta de P frente a NP, en este lenguaje, tiene una forma suficiente: que la unión de las islas cubra todas las CNF. La Proposición 4.8 ya da una respuesta negativa para la isla DSOP. Para las demás no se conoce la respuesta, y ahí empieza el programa. Conviene separar dos conjeturas incompatibles, porque una de las dos tiene que ser falsa.

Conjetura 7.4 (Universalidad afín, riesgo alto)**.** Toda CNF de n variables y m cláusulas admite una ADSOP de tamaño $\mathrm{poly}(n, m)$ construible en tiempo $\mathrm{poly}(n, m)$. Si esta conjetura es cierta, entonces $\#\mathsf{SAT} \in \mathsf{FP}$ y $\mathsf{P} = \mathsf{NP}$.

Conjetura 7.5 (Separación afín)**.** Existe una familia de CNF con $m = \mathrm{poly}(n)$ cuya ADSOP mínima tiene tamaño $2^{\Omega(n)}$. Si esta conjetura es cierta, la ruta afín queda cerrada como ruta universal, igual que la ruta DSOP.

No tenemos una prueba de ninguna de las dos. La evidencia experimental de la Sección 10.3 es ambigua: la ADSOP comprime la paridad y los subespacios afines, pero no una 3-CNF aleatoria. Esa evidencia es compatible con la separación afín y también con una universalidad que requiera una construcción más inteligente que la del prototipo. El valor del par de conjeturas es lógico: fijan exactamente qué habría que demostrar para que la ruta afín resolviera $\mathsf{P} = \mathsf{NP}$ y qué habría que demostrar para cerrarla.

Conjetura 7.6 (Puerta trasera afín)**.** Existe una familia de 3-CNF con $m = O(n)$ que no admite una puerta trasera afín de tamaño $O(\log n)$; de hecho, las 3-CNF aleatorias cerca del umbral no la admiten con probabilidad $1 - o(1)$.

La Conjetura 7.6 es la versión cuantitativa de la ruta de la puerta trasera afín: si toda CNF tuviera una puerta trasera de tamaño $O(\log n)$, el conteo se reduciría a $\mathrm{poly}(n)$ sistemas lineales y tendríamos un algoritmo polinómico. El artículo no la demuestra; la formula porque convierte una intuición práctica en un enunciado falsable. La evidencia de las 3-CNF aleatorias es indirecta: el prototipo ADSOP no las comprime, pero eso no prueba la inexistencia de una puerta trasera pequeña.

7.11. Las islas no son una clasificación vacía

La Tabla 3 muestra que las islas contienen familias no triviales. La cláusula ancha es DSOP de tamaño uno; la paridad es ADSOP de tamaño uno y DSOP exponencial; una CNF de treewidth acotado cae en el conteo o en la ZDD; una CNF con una puerta trasera afín pequeña cae en el conteo afín. Lo que no se sabe es si esas islas, juntas, cubren todas las CNF. La evidencia experimental de la Sección 11 sugiere que las 3-CNF aleatorias cerca del umbral quedan fuera de las islas conocidas, pero una ausencia de compresión experimental no es una cota inferior.

7.12. Paralelización y el modelo de coste

Una objeción natural es la siguiente: si las consultas del motor son paralelizables, ¿no se puede paralelizar también la compilación o el conteo y obtener así un algoritmo polinómico para SAT? La objeción confunde profundidad con trabajo. Un algoritmo paralelo que usa $T(N)$ tiempo y $P(N)$ procesadores realiza $W(N) = O(T(N)P(N))$ operaciones. La simulación secuencial paso a paso ejecuta, en cada paso, las operaciones de los $P(N)$ procesadores; cuesta $O(W(N))$ tiempo.

Proposición 7.7 (Simulación secuencial)**.** *Sea A un algoritmo paralelo uniforme que, para entradas de tamaño N, usa tiempo $T(N)$ y $P(N)$ procesadores. Entonces A se simula secuencialmente en tiempo $O(T(N)P(N))$. En particular, si $T(N)P(N) = \mathrm{poly}(N)$, el problema resuelto por A está en P; el caso NC es $T(N) = \mathrm{polylog}(N)$ y $P(N) = \mathrm{poly}(N)$.*

Demostración. La simulación recorre los pasos $t = 1, \ldots, T(N)$; en el paso t ejecuta secuencialmente las operaciones de los $P(N)$ procesadores sobre la memoria compartida. Cada operación se simula en tiempo $O(1)$ en el modelo PRAM estándar, de modo que el total es $O(T(N)P(N))$. Si $T(N)P(N)$ es polinómico, la simulación es polinómica; el caso NC es un caso particular. Esta

es la inclusión $\mathsf{NC} \subseteq \mathsf{P}$: NC es la clase de los problemas que admiten circuitos uniformes de tamaño polinómico y profundidad polilogarítmica, equivalentemente algoritmos PRAM uniformes de tiempo polilogarítmico y procesadores polinómicos [19]. □

Corolario 7.8 (La paralelización total sí implicaría el colapso)**.** *Si el motor completo —esto es, la compilación DSOP o el contador exacto junto con la fase de consulta— fuera un algoritmo uniforme en* NC *para toda CNF, entonces* $\#\mathsf{SAT} \in \mathsf{P}$ *y* $\mathsf{P} = \mathsf{NP}$*.*

Demostración. Por la Proposición 7.7, la simulación secuencial del algoritmo NC es polinómica. El motor calcularía $\#\mathsf{SAT}$ en tiempo polinómico para toda CNF. Como $\#\mathsf{SAT}$ es $\#\mathsf{P}$-completo [14], $\#\mathsf{P} \subseteq \mathsf{FP}$; por Toda [15], $\mathsf{PH} = \mathsf{P}$. En particular, SAT se reduce a $\#\mathsf{SAT}$ por la comparación con cero, de modo que $\mathsf{P} = \mathsf{NP}$. □

El corolario confirma la intuición: si *todo* el motor fuera paralelizable con trabajo polinómico, el colapso se seguiría. Lo que el artículo no proporciona es esa paralelización total. La fase de consulta es paralelizable en trabajo, pero la compilación y el conteo son las partes caras.

Dada una DSOP de tamaño K, rank se reduce a sumas de K contadores y está en NC en n y K. select se obtiene por búsqueda binaria con $O(n)$ consultas de rank; la búsqueda añade un factor n a la profundidad, de modo que el algoritmo descrito no es necesariamente NC, pero su trabajo total es $O(n^2K)$. Si K es polinómico en el tamaño de la entrada, el trabajo es polinómico y el motor está en P; esto es exactamente el Corolario 5.4. La paralelización no elimina la condición "dado el certificado"; solo distribuye el trabajo.

Si K es exponencial, el trabajo es exponencial. Por la relación $W = T \cdot P$, para ejecutarlo en tiempo polilogarítmico se necesitan $P = W/\operatorname{polylog}(N)$ procesadores, que también es exponencial; eso no es NC. La construcción de la DSOP tiene trabajo proporcional al número de piezas T de COVERTRACE; si T es exponencial, el trabajo es exponencial. La salida misma tiene K cubos; si K es exponencial, ningún algoritmo puede producirla con trabajo polinómico, paralelo o secuencial. La separación fuerte entre CNF de tamaño polinómico y DSOP de tamaño polinómico ya no es un problema abierto: la Proposición 4.8 la establece. Lo que sigue abierto es la separación para ADSOP, ZDD y Fourier.

La reducción del comparador está en NC: el comparador se puede construir con $O(n)$ compuertas y profundidad $O(\log n)$ mediante un prefijo paralelo. Pero la reducción llama al contador A; su paralelización no reduce el trabajo de A. Si A tiene trabajo $2^{o(n)}\operatorname{poly}(n, m)$, la composición hereda ese trabajo; si tuviera trabajo polinómico, ya daría $\#\mathsf{SAT} \in \mathsf{P}$ y $\mathsf{P} = \mathsf{NP}$. La reducción distribuye la llamada, no resuelve el oráculo.

La ZDD ilustra la misma separación: la consulta tiene trabajo $O(n)$ dada la ZDD, pero la construcción directa desde la CNF es el problema abierto. Una construcción paralela con trabajo polinómico sería, por la Proposición 7.7, una construcción secuencial polinómica; entonces el motor daría $\#\mathsf{SAT} \in \mathsf{P}$ y el colapso se seguiría. El artículo no proporciona esa construcción.

La consulta puntual es NC, pero calcula un solo bit. Calcular los 2^n bits en paralelo con 2^n procesadores tiene trabajo 2^n; la paralelización no crea información agregada. Del mismo modo, una búsqueda exhaustiva de las 2^n asignaciones con 2^n procesadores resolvería SAT en tiempo $O(nm)$, pero con trabajo $2^n \cdot nm$; eso no es un algoritmo polinómico en el modelo de P o NC. El modelo cuenta el trabajo total, no solo el tiempo de pared. Permitir hardware exponencial no demuestra $\mathsf{P} = \mathsf{NP}$; cambia el modelo de coste.

En resumen, la paralelización no elimina los argumentos de esta sección: los refuerza. Dado un certificado de tamaño polinómico, el motor tiene consultas de trabajo polinómico; rank es una suma paralelizable y está en NC, mientras que select conserva una búsqueda secuencial y añade un factor n a la profundidad, aunque su trabajo sigue siendo polinómico. La dificultad está en construir el certificado o en contar modelos, y esa dificultad se mide en trabajo total, no en profundidad. Si el artículo hubiera dado un algoritmo NC para $\#\mathsf{SAT}$ para toda CNF, la

simulación secuencial daría un algoritmo polinómico y entonces sí se seguiría $\mathsf{P} = \mathsf{NP}$; pero el artículo no da ese algoritmo.

7.13. Qué se necesitaría para un colapso

El motor no implica $\mathsf{P} = \mathsf{NP}$ porque ninguna de sus rutas proporciona un algoritmo uniforme polinómico para #SAT. Para obtener el colapso haría falta una de estas cinco cosas: un compilador DSOP uniforme polinómico en el tamaño de la entrada y con salida polinómica, un contador exacto polinómico, un compilador ZDD uniforme polinómico en el tamaño de la entrada y con salida polinómica, un compilador ADSOP uniforme polinómico, o un espectro de Walsh–Hadamard de tamaño polinómico construido en tiempo polinómico. Todas implican $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$. La primera está descartada como ruta universal por la Proposición 4.8; quedan cuatro. El artículo demuestra que las consultas son polinómicas una vez que una de esas estructuras está disponible; no demuestra que la estructura esté disponible en tiempo polinómico. Esa es la frontera exacta: el motor es una capa de consulta, no un colapso. Como muestra la Sección 7.12, la paralelización no cambia esta conclusión: un algoritmo NC para todo el motor implicaría $\mathsf{P} = \mathsf{NP}$, pero el artículo no lo proporciona.

La distinción no es una formalidad. Si se confundiera "consultar una DSOP" con "compilar una DSOP", se obtendría la hipótesis del Teorema 7.1 por definición y se concluiría $\mathsf{PH} = \mathsf{P}$. El error sería el mismo que confundir un verificador de certificados con un buscador de certificados. El motor es una capa de consulta y verificación; la búsqueda del certificado es el problema abierto.

7.14. Programa hacia $\mathsf{P} = \mathsf{NP}$

El mapa anterior sugiere un programa con cinco frentes. No es una lista de deseos: cada frente tiene un enunciado falsable y, en varios casos, una consecuencia inmediata para el colapso.

(P1) *Cotas inferiores para ADSOP.* Demostrar o refutar la Conjetura 7.5. Una cota de $2^{\Omega(n)}$ para una familia de tamaño polinómico cerraría la ruta afín; el experimento con 3-CNF aleatorias es evidencia, no prueba.

(P2) *Puertas traseras afines.* Demostrar o refutar la Conjetura 7.6. Es el frente más atacable computacionalmente: se pueden buscar puertas traseras en instancias pequeñas y medir su crecimiento.

(P3) *Compilación directa a ZDD y Fourier.* Construir esas estructuras sin pasar por la DSOP. Una cota inferior o un compilador para una familia rica decidiría la ruta.

(P4) *Complejidad de pruebas.* Comparar K con resolución, *cutting planes* y *polynomial calculus*; la Proposición 4.8 es un primer dato.

(P5) *Un compilador universal o un contador polinómico.* Es el objetivo final; debe ser uniforme, determinista, con salida polinómica y para toda CNF.

El programa se puede resumir en dos extremos. En un extremo, un compilador universal polinómico para la ruta afín (Conjetura 7.4) daría $\mathsf{P} = \mathsf{NP}$. En el otro, una familia afín-incompresible (Conjetura 7.5) cerraría la ruta. Entre ambos quedan las situaciones intermedias: una cota inferior superpolinomial pero no exponencial, una universalidad solo para familias estructuradas, o una representación distinta. El artículo no resuelve ninguno de los extremos; los formula para que el trabajo siguiente sepa qué resultado contaría como avance. Algo análogo vale para ZDD y Fourier.

Las conjeturas tienen distinto estatus. La Conjetura 7.4 es la más fuerte del artículo y no tenemos evidencia directa a su favor; la incluimos porque fija el objetivo lógico. La Conjetura 7.5 es más plausible a la luz de las 3-CNF aleatorias, pero también está abierta. El programa no consiste en elegir una de las dos: consiste en encontrar la primera familia o el primer compilador que decida la cuestión.

Ningún resultado de este artículo depende de las Conjeturas 7.4–7.6. Se incluyen como un programa de investigación, separadas de los teoremas y de las proposiciones demostradas.

8. Algoritmo híbrido

Teorema 8.1 (Corrección del motor híbrido)**.** *Cada rama del Algoritmo 1 que termina con éxito devuelve los valores correctos de* bit, rank_0, rank_1, select_0, select_1, #SAT *y muestreo uniforme para* $S(F)$.

Demostración. La rama materializada usa la definición del bitvector. La rama DSOP usa el Teorema 5.3. La rama de conteo usa el Teorema 6.2. La función bit es la Proposición 3.1 en las ramas no materializadas y la definición del bitvector en la materializada. La búsqueda binaria y el muestreo son correctos por la monotonía de rank_0 y la biyección $R \mapsto \text{select}_0(R)$. □

Algoritmo 1 Motor de consultas para $S(F)$

Require: CNF F con n variables y m cláusulas; umbrales $N_0, K_{\max}$; contador exacto A (opcional)
Ensure: Operaciones $\text{bit}, \text{rank}_0, \text{rank}_1, \text{select}_0, \text{select}_1, \#\mathsf{SAT}$ y muestreo

```
if n ≤ N_0 then
    materializar S(F) y construir un bitvector con rank/select sucinto
else
    ejecutar COVERTRACE con tope K_max
    if éxito y K ≤ K_max then
        construir el motor DSOP del Teorema 5.3
    else if A está disponible then
        usar el Teorema 6.2 para rank y búsqueda binaria para select
    else
        devolver NoDisponible para consultas agregadas
    end if
end if
función bit(k):
if bitvector materializado then
    devolver el bit k de S(F) {O(1)}
else
    devolver 1 − F(assign(k)) {O(m)}
end if
```

El tope $K_{\max}$ limita el tamaño de la salida de COVERTRACE; no garantiza que la búsqueda sea polinómica. El algoritmo selecciona representaciones; no acota el coste de compilación. La corrección es condicional al éxito de la rama, y la Sección 7 analiza por qué esa distinción impide un colapso de clases.

Consulta	DSOP (K cubos)	Contador exacto A	Materializada
$\text{bit}(k)$	$O(m)$	$O(m)$	$O(1)$
#SAT	$O(K)$	$T(n+O(n), m+O(n))$	$O(2^n/w)$
$\text{rank}(k)$	$O(nK)$	$T(n+O(n), m+O(n))$	$O(1)$
$\text{select}(r)$	$O(n^2K)$	$O(n\,T(n+O(n), m+O(n)))$	$O(1)$
muestreo	$O(n^2K)$	$O(n\,T(n+O(n), m+O(n)))$	$O(1)$

Tabla 2: Trabajo por consulta; el preprocesamiento de la DSOP y el del bitvector no se incluyen. w es el tamaño de palabra. La columna A es condicional a la disponibilidad del contador exacto.

Observación 8.2 (Trabajo y profundidad)**.** Las cotas de la Tabla 2 son de trabajo por consulta una vez construida la representación; no son cotas de profundidad paralela. En un modelo PRAM, bit tiene profundidad $O(\log m)$ con una reducción sobre las cláusulas; #SAT y rank son sumas y prefijos con profundidad $O(\log n + \log K)$; select conserva la búsqueda binaria de n pasos y, aun paralelizando rank y calculando los conteos de prefijo como prefijos paralelos, su profundidad es $O(n(\log n + \log K))$ con el mismo trabajo $O(n^2 K)$. La rama del contador hereda la profundidad D_A de A y el factor n de la búsqueda, de modo que su profundidad es $O(n(D_A + \log n))$; la rama materializada paga $O(m2^n)$ de trabajo de construcción, aunque sus consultas sean $O(1)$. La subsección 7.12 demuestra que un algoritmo NC para el motor completo implicaría P = NP; paralelizar la fase de consulta de una representación dada no cambia las cotas de trabajo ni la condición de colapso.

Las cotas de la columna DSOP son operaciones aritméticas sobre contadores de $O(n)$ bits, salvo bit, que se mide en evaluaciones de cláusula; en el modelo de bits se multiplican por $O(n/\log n)$. La columna materializada supone un índice sucinto de *rank*/*select*. La Tabla 2 resume el compromiso. La rama materializada solo es viable para n pequeño; la rama DSOP es la mejor cuando la compilación es compacta; la rama de conteo es la que puede responder para una k-CNF con DSOP exponencial, a costa de depender de un contador exacto y de pagar una consulta de conteo por cada consulta de rank.

Observación 8.3 (El bit en las ramas comprimidas)**.** En las ramas DSOP y contador, bit(k) se responde evaluando F; no hay que construir la DSOP ni llamar al contador para una consulta puntual. En la rama materializada, basta leer el bit k del bitvector. Así, una consulta puntual no obliga a pagar la compilación de la DSOP ni una llamada al contador; en la rama materializada, el coste $O(1)$ presupone que el bitvector ya se construyó para otras consultas.

8.1. Qué garantiza el tope $K_{\max}$

El tope $K_{\max}$ acota el tamaño de la salida de COVERTRACE, no el tiempo de búsqueda. Si COVERTRACE tiene éxito y produce $K \le K_{\max}$, el Teorema 5.3 da consultas polinómicas en n y K; pero el tiempo para encontrar esa DSOP no está acotado por $K_{\max}$ y puede ser exponencial. Algo análogo ocurre con las otras ramas: la materializada tiene consultas $O(1)$ pero un preprocesamiento de $O(m2^n)$, y la del contador hereda el coste de A.

No se garantiza una cota polinómica: no hay un teorema que asegure que la búsqueda de la DSOP o la llamada al contador termine en tiempo polinómico para toda CNF. Tampoco se demuestra que el algoritmo sea exponencial en el peor caso; la dureza #P es del problema exacto, no de cada rama del híbrido. Una ausencia de cota superior no implica una cota inferior, y una cota inferior para el problema no se traduce en una para cada instancia.

El Algoritmo 1 es un algoritmo *parcial*: puede devolver NoDisponible. No es un algoritmo uniforme que resuelva #SAT para toda CNF; por eso no contradice #P ni H-ETH. Si se completara con una enumeración en la rama NoDisponible, el peor caso sería exponencial bajo H-ETH, pero esa completación no forma parte del motor. La corrección es condicional al éxito de la rama, como ya afirma el Teorema 8.1.

8.2. Por qué el híbrido no es inevitablemente exponencial

P ≠ NP no implica que toda instancia de SAT sea difícil; implica que no hay un algoritmo uniforme polinómico para todas. Algo análogo vale para #SAT: la dureza #P y H-ETH son afirmaciones de peor caso. Una familia con DSOP polinómica y compilable en tiempo polinómico, ancho de árbol acotado, puerta trasera pequeña, o una ADSOP de tamaño polinómico y compilable en tiempo polinómico, puede resolverse en tiempo polinómico; el motor aprovecha la

representación cuando está disponible. La misma CNF puede ser fácil para una representación y difícil para otra; el híbrido elige.

Hay, por tanto, tres niveles que no deben confundirse:

(I) *Consulta dada una representación.* Con una DSOP de tamaño K es polinómica en n y K (Teorema 5.3); con un contador exacto A cuesta una llamada a A por consulta de rank. No depende de $\mathsf{P} = \mathsf{NP}$.

(II) *Construcción de la representación o del contador.* Es específica de la instancia y de la familia; puede ser polinómica (cláusula ancha, DSOP pequeña, treewidth acotado, ADSOP afín) o exponencial. No hay un compilador uniforme polinómico conocido; uno así daría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$.

(III) *Problema exacto en el peor caso.* Es $\#\mathsf{P}$-difícil; bajo H-ETH o $\#\mathsf{ETH}$ no hay algoritmo $2^{o(n)}$ para toda CNF. Esto no dice que cada instancia sea difícil, ni que el híbrido concreto sea exponencial en cada familia.

El motor es un *portafolio*: intenta varias representaciones y paga el coste del intento más el de la rama que tiene éxito. Si una rama es polinómica en una familia y el intento también lo es, el híbrido es polinómico en esa familia, aunque el peor caso uniforme siga siendo exponencial.

Las mejoras que no se ven en la cota teórica son reales: mejor orden de variables, preprocesamiento, descomposición en componentes, caché, estructuras sucintas, paralelización, mejores solvers SAT y mejores compiladores pueden reducir el trabajo por un factor polinómico o cambiar el régimen práctico sin cambiar la cota de peor caso. La teoría dice qué ramas son polinómicas en sus parámetros y qué barreras existen; no predice el tiempo en una instancia concreta. Por eso la evaluación del motor es experimental: mide escalado, cruces y tasa de éxito de las ramas, y no infiere una cota de complejidad a partir de unos benchmarks.

Una mejora puede ser teórica para una subclase —K polinómico, treewidth acotado, puerta trasera pequeña, ADSOP para paridad— y experimental para el resto. El artículo debe presentar ambas cosas por separado: los teoremas condicionales y las barreras, por un lado; el portafolio y su evaluación, por otro. La pregunta "¿es inevitablemente exponencial?" tiene una respuesta graduada: no para toda instancia, sí como peor caso uniforme bajo las conjeturas estándar si se exige un algoritmo total, y no se conoce una cota uniforme para el híbrido concreto; en cada familia debe analizarse o medirse.

8.3. Condiciones de éxito por rama

La pregunta útil no es "¿es el motor polinómico para toda CNF?" sino "¿qué familias hacen que una rama sea polinómica y cuánto cuesta decidir que la instancia pertenece a esa familia?". La proposición siguiente formaliza la condición suficiente.

Proposición 8.4 (Condición suficiente por familia)**.** *Sea $\mathcal{F}$ una familia de CNF y R una representación con parámetro $p_R(F)$. Si existen*

(I) un test de pertenencia a $\mathcal{F}$ en tiempo $\mathrm{poly}(n, m)$*;*
(II) un compilador que produce $R(F)$ en tiempo $\mathrm{poly}(n, m, p_R(F))$*;*
(III) una cota $p_R(F) \le \mathrm{poly}(n, m)$ para toda $F \in \mathcal{F}$,

entonces el motor responde $\mathrm{bit}, \#\mathsf{SAT}, \mathrm{rank}, \mathrm{select}$ *y muestreo en tiempo* $\mathrm{poly}(n, m)$ *para toda $F \in \mathcal{F}$. En la rama de contador, R denota el contador A y (ii) se interpreta como disponibilidad de A con coste T.*

Demostración. El test elige la rama; el compilador produce $R(F)$; las consultas son polinómicas en n y $p_R(F)$ por los Teoremas 5.3 y 6.2 y por la definición de la rama materializada o estocástica. Componiendo las tres cotas se obtiene el resultado. Si $\mathcal{F}$ es una clase con promesa, el test (i) se omite; si el motor debe decidir si la instancia pertenece a la familia, (i) forma parte del coste. □

Observación 8.5 (La familia propia es esencial)**.** Si $\mathcal{F}$ es toda CNF y se cumplen (i)–(iii) para una representación exacta o un contador exacto, entonces $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$. Por tanto, el programa de condiciones de éxito debe producir familias propias o parámetros; cubrir todas las CNF sin un colapso no es posible con una representación exacta y un test polinómico.

La Tabla 3 resume las ramas, sus parámetros y las condiciones suficientes conocidas. La columna de consulta es trabajo *dado el testigo*; no incluye el coste de encontrarlo.

Rama	Testigo / parámetro	Consulta (trabajo)	Condición suficiente	Cuello / qué falta
Materializada	$S(F)$	$O(1)$	$n \le N_0$	Construcción $O(m2^n)$.
DSOP	K cubos	$O(nK)$, $O(n^2K)$	$K \le$ poly; treewidth/pathwidth acotado, pocas cláusulas, backdoor pequeño	COVERTRACE y verificación de cobertura.
Contador A	coste T	T, nT	A eficiente en la familia (afín, treewidth, backdoor)	Un contador polinómico para toda CNF colapsaría.
ZDD	tamaño Z	$O(n)$	$Z \le$ poly; pathwidth/treewidth acotado	Construir la ZDD y elegir el orden.
Fourier	sparsity s	$O(sn)$, $O(sn^2)$	$s \le$ poly; paridad, junta, espectro disperso	Construir o detectar el espectro.
ADSOP	A piezas afines	$O(A)$, $O(n^4A)$	$A \le$ poly; paridad, subespacios afines, backdoor afín	Compilador y cota inferior.
Estocástico	ε, δ	aproximado	densidad no minúscula; hashing	Exactitud; cancelación del complemento.

Tabla 3: Ramas, parámetros y condiciones suficientes. La consulta es trabajo dado el testigo; el coste de encontrar el testigo o de decidir la pertenencia no está incluido.

La tabla separa dos problemas que suelen confundirse: (a) dada la representación, la consulta es polinómica; (b) decidir que la instancia está en la familia y construir la representación puede ser difícil. Una condición de éxito no es un algoritmo hasta que el test y el compilador sean polinómicos. La investigación útil consiste en ampliar las familias y abaratar el test, no en postular que toda CNF está en una. La paridad es el caso testigo: la DSOP es exponencial, pero la ADSOP, el espectro y un contador afín son polinómicos; una cláusula ancha es DSOP de tamaño uno; una CNF de treewidth acotado cae en el contador o en la ZDD; una 3-CNF aleatoria cerca del umbral puede quedar fuera de todas las condiciones conocidas, aunque en n pequeño el tamaño de $U(F)$ puede hacer que la DSOP de U sea pequeña.

Piloto de regímenes. El script `branch_regime_map.py` calcula los parámetros de la Tabla 4 en $n = 10$. No mide tiempos de consulta; su función es comprobar que las ramas no son una clasificación vacía: la paridad tiene $K = 512$ y ADSOP 1; la cláusula ancha tiene $K = 1$; la cadena tiene treewidth estimado 2; la 3-CNF con $r = 2$ no tiene rama exacta barata, mientras que las instancias con $r = 3$ y $r = 4.2$ tienen pocas soluciones y su región prohibida es casi todo el cubo, lo que reduce K.

Familia	m	vol(U)	#SAT	K	A	s	tw	Ramas viables
ancha	1	32	992	1	1	32	4	DSOP, ADSOP, Fourier, A
paridad	512	512	512	512	1	2	9	ADSOP, Fourier, A afín
cubo+paridad	768	768	256	512	2	4	9	ADSOP, Fourier, A afín
3-CNF $r = 2$	20	943	81	285	281	1024	6	estocástico
3-CNF $r = 3$	30	1019	5	38	38	1024	7	DSOP, ADSOP, estocástico
3-CNF $r = 4.2$	42	1021	3	25	25	1024	7	DSOP, ADSOP, estocástico
cadena	8	520	504	306	144	586	2	A treewidth, estocástico
componentes	3	338	686	162	40	512	2	ADSOP, A componentes, estocástico

Tabla 4: Piloto de regímenes en $n = 10$. K y A son las salidas de `cover` y `affine_cover`; tw es una estimación por eliminación mínima; la rama materializada se omite porque $n \leq N_0$. No se miden tiempos ni escalado.

Redes tensoriales y rango. Una rama adicional, todavía teórica, es la contracción de una red tensorial con cortes de rango bajo. El parámetro es el rango de corte o el ancho de contracción; la consulta es una contracción; la condición suficiente es que ese rango sea polinómico; el cuello es encontrar la descomposición. El manuscrito de Anthropic [6] usa una jerarquía de cortes, una identidad de ocho cuadrados y un equilibrio de bits para obtener un algoritmo $2^{o(n)}$ para #k-SAT. No es una rama práctica: no hay implementación, la constante no es explícita y el propio manuscrito descarta mejoras hasta $n > 10^{10\,000}$ en el caso $m = 5n$. Tampoco sustituye a un compilador de DSOP o ADSOP: su salida es un contador subexponencial, no una representación que responda rank o select.

9. Optimización mediante ZDD y Fourier

La estructura del Teorema 5.3 responde rank en $O(nK)$ porque suma los prefijos de K cubos. Una optimización natural es compilar la familia disjunta en un *Zero-suppressed Decision Diagram* (ZDD) ordenado [11, 10], con el orden de variables $x_1, \ldots, x_n$, en el que cada nodo almacena el número de asignaciones de su subgrafo. Convenimos que la ZDD representa la familia $U(F)$ de asignaciones que falsifican F; el número de conjuntos de la ZDD es $|U(F)|$ y $\#\mathsf{SAT}(F) = 2^n - |U(F)|$. En una ZDD, bit, rank, select y el muestreo se responden recorriendo a lo sumo n niveles, una vez construida la estructura.

Si la ZDD se construye desde la DSOP de K cubos, su tamaño es $O(nK)$: el trie de los K patrones tiene a lo sumo nK nodos y la reducción solo puede disminuirlo. La ventaja es que las consultas bajan de $O(nK)$ a $O(n)$. La pregunta interesante es si puede construirse directamente desde la CNF una ZDD de tamaño mucho menor que nK, sin pasar por la DSOP. La paridad es el caso testigo: su DSOP mínima tiene 2^{n-1} cubos, pero su ZDD tiene $O(n)$ nodos, porque la familia se describe por una ecuación lineal. Construir esa ZDD directamente desde la CNF es un problema abierto (Sección 13).

La ZDD es, por tanto, una pieza de ingeniería dentro del motor híbrido: si la estructura comprimida es pequeña, las consultas bajan a $O(n)$; si no, se conserva el motor DSOP o se usa el oráculo de conteo.

9.1. Representación de Fourier

La tabla de verdad $S(F)$ también admite una representación por su espectro de Walsh–Hadamard [8]. Sea $f(x) = S(F)[x]$. Para $S \subseteq [n]$, el coeficiente de Fourier es

$$\hat{f}(S) = \sum_{x \in \Omega_n} f(x)\,(-1)^{\sum_{i \in S} x_i}.$$

La transformada inversa es

$$f(x) = \frac{1}{2^n} \sum_{S \subseteq [n]} \hat{f}(S)\,(-1)^{\sum_{i \in S} x_i}.$$

El coeficiente vacío es $\hat{f}(\varnothing) = |U(F)|$, de modo que $\#\mathsf{SAT}(F) = 2^n - \hat{f}(\varnothing)$. Sea $s = |\{S \subseteq [n] : \hat{f}(S) \neq 0\}|$ la *densidad* del espectro.

Si los s coeficientes no nulos están disponibles, $\mathrm{rank}_1(k)$ se calcula como

$$\mathrm{rank}_1(k) = \frac{1}{2^n} \sum_{S : \hat{f}(S) \neq 0} \hat{f}(S) \sum_{\mathrm{pos}(x) < k} (-1)^{\sum_{i \in S} x_i}.$$

La suma interna sobre el prefijo se calcula en $O(n)$ con el autómata de dos estados del Lema 5.1, adaptado a caracteres. Por tanto, rank cuesta $O(sn)$, select cuesta $O(sn^2)$ por búsqueda binaria y el muestreo $O(sn^2)$. El motor de Fourier es exacto: los coeficientes son enteros y la división por 2^n se acumula al final.

La representación es útil cuando s es pequeño. La paridad tiene $s = 2$: $S(F_n)$ es el indicador de paridad impar y su espectro se concentra en $\varnothing$ y $[n]$, con coeficientes 2^{n-1} y -2^{n-1}. En cambio, una cláusula ancha de ancho w tiene $s = 2^w$: su cubo falsificador fija w coordenadas y el espectro se distribuye sobre los subconjuntos de esas coordenadas. Una función booleana aleatoria tiene un espectro denso: la mayoría de los 2^n coeficientes son no nulos. No hay, por tanto, una garantía universal de compresión.

El algoritmo de [7] acelera la DFT de longitud N en un factor polilogarítmico, pero para $N = 2^n$ sigue siendo exponencial en n; no sustituye a un compilador de espectro disperso. La pregunta relevante para el motor es si el espectro de $S(F)$ puede construirse con s polinómico en tiempo polinómico; el coeficiente vacío ya es $|U(F)|$, así que construir el espectro para toda CNF es al menos tan difícil como contar modelos.

La representación de Fourier es, por tanto, una tercera pieza de compresión, complementaria a la DSOP y a la ZDD: gana en la paridad y en funciones de espectro disperso, y pierde en cláusulas anchas o funciones aleatorias. La Sección 7 la incluye como una ruta más hacia el colapso.

10. Oráculos estocásticos y coberturas afines

Las Secciones 5–9 suponen que la DSOP, la ZDD o el espectro ya están construidos. Esta sección aborda la fase de construcción desde dos ángulos adicionales. El primero es estocástico: se sustituye la respuesta exacta por una muestra o por un estimador con error controlado. El segundo es algebraico: se generalizan los cubos a subespacios afines sobre $\mathbb{F}_2$, de modo que la paridad deja de ser el testigo negativo de la DSOP. Ninguna de las dos familias elimina la barrera $\#\mathsf{P}$; ambas desplazan el coste y cambian el tipo de garantía.

10.1. Tipos de oráculo y separaciones

Para una asignación parcial $v \in \{0, 1, \bullet\}^n$, sea $Q(v)$ el subcubo de las asignaciones que extienden v, y sea $F|_v$ la CNF obtenida al fijar las variables de v; así $U(F|_v) = Q(v) \cap U(F)$. Una interfaz de consulta sobre el cubo puede ofrecer cinco operaciones distintas:

- $\mathrm{bit}(x)$: el valor $f(x)$ en una asignación completa; es local y cuesta $O(m)$;
- $\mathsf{has1}(v)$: si $Q(v) \cap U(F) \neq \emptyset$; es una consulta de satisfacibilidad;
- $\mathsf{sample}(v)$: una muestra uniforme de $Q(v) \cap U(F)$;
- $\mathsf{est}(v, \varepsilon, \delta)$: una estimación multiplicativa de $\mathrm{vol}(Q(v) \cap U(F))$;

- $\mathsf{count}(v)$: el volumen exacto $\mathrm{vol}(Q(v) \cap U(F))$.

La Tabla 5 resume qué se puede obtener de cada uno. La diferencia entre $\mathsf{has1}$ y count es la diferencia entre decidir y contar; la diferencia entre sample y count es la diferencia entre una distribución y una normalización.

Interfaz	Respuesta	Testigo	#SAT exacto	#SAT aproximado	rank / select
$\mathrm{bit}(x)$	$f(x)$	no global	2^n consultas	solo si la densidad es alta	no
$\mathsf{has1}(v)$	$Q(v) \cap U(F) \neq \emptyset$	$O(n)$	no conocido	$\mathsf{BPP}^{\mathsf{NP}}$	no exacto
$\mathsf{sample}(v)$	uniforme en $Q(v) \cap U(F)$	$O(1)$	no	implica contar aprox.	aproximado
$\mathsf{est}(v, \varepsilon, \delta)$	$(1 \pm \varepsilon)$ con confianza	no	no	sí	aproximado
$\mathsf{count}(v)$	$\mathrm{vol}(Q(v) \cap U(F))$	$O(n)$	una consulta	sí	sí
WHT disperso	coeficientes	sí	si s es poly	sí	sí
ADSOP	unión disjunta de afines	sí	$O(K)$	sí	sí

Tabla 5: Tipos de interfaz y qué garantía ofrece cada uno. La columna #SAT exacto se refiere a un algoritmo uniforme para toda CNF; la aproximada es con error relativo.

Proposición 10.1 (La decisión no da el conteo exacto)**.** *Si existiera un algoritmo determinista polinómico con oráculo* $\mathsf{has1}$ *que calculara* $\#\mathsf{SAT}(F)$ *para toda CNF, entonces* $\#\mathsf{P} \subseteq \mathsf{FP}^{\mathsf{NP}}$ *y, por el teorema de Toda,* $\mathsf{PH} = \Delta_2^{\mathsf{P}} = \mathsf{P}^{\mathsf{NP}}$.

Demostración. El algoritmo daría $\#\mathsf{SAT} \in \mathsf{FP}^{\mathsf{NP}}$. Por la completitud de #SAT para #P, se sigue $\#\mathsf{P} \subseteq \mathsf{FP}^{\mathsf{NP}}$ y, por tanto, toda máquina polinómica con un oráculo #P se simula con un oráculo NP, es decir, $\mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}^{\mathsf{NP}}$. Por Toda, $\mathsf{PH} \subseteq \mathsf{P}^{\#\mathsf{P}} \subseteq \mathsf{P}^{\mathsf{NP}} = \Delta_2^{\mathsf{P}} \subseteq \mathsf{PH}$, de donde $\mathsf{PH} = \Delta_2^{\mathsf{P}}$. □

Observación 10.2 (Aproximar no es contar)**.** El conteo aproximado con error relativo y confianza es posible con un algoritmo probabilístico polinómico que consulta un oráculo NP (Stockmeyer [21]); no se conoce un algoritmo exacto polinómico. No se sabe que $\mathsf{P} = \mathsf{NP}$ implique $\#\mathsf{SAT} \in \mathsf{FP}$. Un compilador exacto polinómico sí implicaría $\#\mathsf{SAT} \in \mathsf{FP}$, pero no se conoce una implicación en el sentido opuesto.

10.2. Muestreo, hashing y la cancelación de Karp–Luby

Muestreo directo. Sea $p = \#\mathsf{SAT}(F)/2^n$. Si se muestrea $x \sim \Omega_n$ y se evalúa $F(x)$, el estimador

$$\hat{p} = \frac{1}{T} \sum_{j=1}^{T} F(x_j), \qquad \widehat{\#\mathsf{SAT}} = 2^n \hat{p}$$

es insesgado. Hoeffding da

$$\Pr\big(|\hat{p} - p| \geq \varepsilon\big) \leq 2e^{-2T\varepsilon^2}.$$

El error aditivo es barato: $T = O(\log(1/\delta)/\varepsilon^2)$. El error relativo es el problema:

$$\frac{\Delta\#\mathsf{SAT}}{\#\mathsf{SAT}} \approx \frac{1}{\sqrt{Tp}}.$$

Si $p = 2^{-n}$ (una sola solución), hacen falta $T \approx 2^n/\varepsilon^2$ muestras. Cuando la densidad es exponencialmente pequeña, el muestreo uniforme requiere un número exponencial de muestras y la cota de Hoeffding deja de ser útil en la práctica.

Hashing XOR. Se añaden t restricciones de paridad aleatorias

$$H_t : \quad \bigoplus_{i \in S_j} x_i = b_j, \qquad j = 1, \ldots, t,$$

con $S_j \subseteq [n]$ aleatorio no vacío y b_j uniforme en $\{0,1\}$. Para una solución fija, cada una de las t paridades aleatorias se satisface con probabilidad $1/2$; por tanto, la solución sobrevive con probabilidad 2^{-t}. El rango de las restricciones no cambia esa probabilidad: solo determina cuántas soluciones tiene el sistema hasheado. Entonces

$$\mathbb{E}\big[\mathrm{count}(F \wedge H_t)\big] = \frac{\#\mathsf{SAT}}{2^t}, \qquad \widehat{\#\mathsf{SAT}}_t = \mathrm{count}(F \wedge H_t)\, 2^t$$

es insesgado. El estimador que implementamos es exactamente $\widehat{\#\mathsf{SAT}}_t$, con count calculado por un solver exacto sobre la CNF hasheada.

La insesgadez no basta para una garantía de error relativo. El análisis de ApproxMC [23] obtiene $(1 \pm \varepsilon)$ con probabilidad $1 - \delta$ bajo una familia de hash t-wise independiente y una elección adaptativa de t que mantiene el conteo esperado en un rango constante; el número de repeticiones y la constante dependen de ε y δ. Nuestro prototipo no implementa la familia t-wise independiente completa ni la elección adaptativa certificada: usa subconjuntos aleatorios, un límite de 64 modelos, siete repeticiones y una heurística para t. Por tanto, la garantía formal es del algoritmo idealizado, no del prototipo. La Tabla 6 reporta el error observado, no una cota de error; un estimador insesgado puede tener error relativo grande, y la mediana de pocas repeticiones no sustituye una prueba de concentración.

La Tabla 6 resume el experimento con 120 000 muestras directas y un prototipo de ApproxMC con límite de 64 modelos y siete repeticiones. Los casos con pocas soluciones ilustran la diferencia: el muestreo directo no detecta la solución con probabilidad útil, mientras que el hashing reduce el espacio y la localiza.

Caso	#SAT	Directo	Error rel.	Hashing t	Hashing	Error rel.
solución única $n = 20$	1	0	100 %	0	1	0 %
disjunto $n = 20$	470 596	472 768	0.46 %	13	471 040	0.09 %
disjunto $n = 24$	5 764 801	5 747 874	0.29 %	17	6 291 456	9.14 %
3-CNF $n = 24$, $m = 72$	1 838	1 957	6.49 %	6	1 984	7.94 %
3-CNF $n = 26$, $m = 78$	3 081	2 237	27.39 %	6	3 328	8.02 %
UNSAT $n = 18$, $m = 75$	0	0	0 %	0	0	0 %

Tabla 6: Muestreo directo frente a hashing XOR. El hashing es aproximado y la tabla reporta el error observado de un prototipo, no una garantía FPRAS. El caso de solución única con $t = 0$ es exacto porque el número de modelos ya es pequeño y se enumera.

Karp–Luby y la cancelación del complemento. Hay una ruta que parece resolver el problema y no lo resuelve. La región prohibida

$$U(F) = \bigcup_{C \in F} Q(p(C))$$

es una DNF con m términos. Karp–Luby [20] estima el tamaño de una unión de conjuntos:

$$N = \sum_{C \in F} \mathrm{vol}(p(C)), \qquad \widehat{|U|} = \frac{1}{T} \sum_{j=1}^{T} \frac{N}{c(x_j)},$$

donde se elige un cubo C con probabilidad $\mathrm{vol}(p(C))/N$, se muestrea x_j uniforme en $Q(p(C))$, y $c(x_j)$ es el número de cubos que contienen x_j. El estimador es insesgado y da un FPRAS de $|U|$ en tiempo $\mathrm{poly}(n, m, 1/\varepsilon, \log(1/\delta))$.

El problema es la resta:

$$\#\mathsf{SAT} = 2^n - |U|.$$

Si $|U|$ se estima con error relativo ε, el error absoluto es $\varepsilon|U|$, y el error relativo de $\#\mathsf{SAT}$ es

$$\frac{\varepsilon|U|}{\#\mathsf{SAT}}.$$

Cuando $\#\mathsf{SAT}$ es una fracción minúscula de 2^n, ese factor es enorme; la estimación puede incluso ser negativa. La Tabla 7 lo muestra.

Caso	$\#\mathsf{SAT}$	$\lvert U\rvert$ estimado	$\#\mathsf{SAT}$ estimado	Error rel.
solución única $n = 20$	1	1 047 980.6	595.4	59 441 %
disjunto $n = 24$	5 764 801	11 031 257.6	5 745 958.4	0.33 %
3-CNF $n = 24$, $m = 72$	1 838	16 797 232.4	−20 016.4	1 189 %
3-CNF $n = 20$, $m = 40$	5 198	1 046 867.7	1 708.3	67.14 %

Tabla 7: Karp–Luby con 50 000 muestras estima bien $|U|$, pero la resta $2^n - |U|$ destruye la precisión relativa de $\#\mathsf{SAT}$ cuando la densidad de soluciones es minúscula.

Observación 10.3 (Contar la DNF prohibida no es contar SAT)**.** El FPRAS de Karp–Luby es un oráculo estocástico para $U(F)$, no para F. La cancelación del complemento es la contrapartida del muestreo directo: en un caso se estima la región de soluciones, que puede ser minúscula; en el otro, la región complementaria, que puede ser casi todo el cubo. El estimador no impone la cota $|U| \le 2^n$; por eso la resta puede producir un valor negativo, como en la Tabla 7. Esto no contradice el FPRAS: el error relativo de $|U|$ es pequeño, pero el de $\#\mathsf{SAT}$ no. El absurdo es la lección, no un error de implementación: una estimación con error relativo pequeño en el complemento puede ser inútil para el conjunto original cuando este es diminuto. Para un FPRAS de $\#\mathsf{SAT}$ haría falta controlar el error absoluto de $|U|$ con precisión $O(\#\mathsf{SAT})$, que en el régimen raro es exponencial.

Particiones, creencia y física estadística. Una partición aleatoria de subcubos con $\mathsf{has1}$ y $\mathsf{has0}$ puede podar cuando el cubo es homogéneo; si el árbol se cierra, el conteo es exacto; si se detiene en hojas mixtas, se obtiene un intervalo de confianza. El número de subcubos distintos está relacionado con el número de cofactores y con el tamaño del OBDD para ese orden. Los métodos de Monte Carlo sobre asignaciones (Metropolis, Gibbs, Wang–Landau) y la propagación de creencias o *survey propagation* [24] estiman marginales o densidades de estados; en árboles son exactos, en grafos con ciclos son heurísticas. Para $\#\mathsf{SAT}$ exacto no sirven; para instancias grandes y densas son una alternativa práctica de estimación.

Subespacios afines aleatorios. Las restricciones XOR son intersecciones con subespacios afines. En vez de muestrear bits, se muestrea un subespacio afín de dimensión $n - r$; esto es exactamente lo que hace el hashing. La diferencia con el muestreo de bits es que el subespacio puede ser exponencialmente grande y, aun así, manejable por álgebra lineal. Es el puente entre la parte estocástica y la cobertura afín de la Sección 10.3.

10.3. Cobertura afín: ADSOP

La DSOP cubre $U(F)$ con cubos, y un cubo es un caso particular de subespacio afín sobre $\mathbb{F}_2$. La generalización natural es una *suma disjunta de subespacios afines* (ADSOP):

$$U(F) = \bigsqcup_{j=1}^{K} A_j, \qquad A_j = p_j + \langle b_{j,1}, \ldots, b_{j,d_j} \rangle_{\mathbb{F}_2},$$

donde $b_{j,1}, \ldots, b_{j,d_j}$ son linealmente independientes. Cada pieza tiene exactamente 2^{d_j} puntos y $d_j = \dim A_j$. La DSOP es el caso en que los $b_{j,\ell}$ son vectores unitarios y p_j fija las demás coordenadas.

Teorema 10.4 (Consultas exactas sobre una ADSOP)**.** *Sea $U(F) = \bigsqcup_{j=1}^{K} A_j$ una ADSOP con $\dim A_j = d_j$ y $A_j = p_j + \langle b_{j,1}, \ldots, b_{j,d_j} \rangle$. Para un patrón v, sea $M_{j,v}$ el sistema lineal en las variables $c_1, \ldots, c_{d_j}$ que impone*

$$\sum_{\ell=1}^{d_j} c_\ell \, (b_{j,\ell})_i = v_i - (p_j)_i \qquad \text{para cada } i \in \operatorname{supp}(v).$$

Entonces:

(I) $|U(F)| = \sum_{j=1}^{K} 2^{d_j}$ *y* $\#\mathsf{SAT}(F) = 2^n - |U(F)|$*;*

(II) un testigo de $U(F)$ se obtiene resolviendo un sistema lineal en una pieza no vacía;

(III) para todo patrón v,

$$|Q(v) \cap A_j| = \begin{cases} 2^{d_j - \operatorname{rk}(M_{j,v})}, & \text{si } M_{j,v} \text{ es compatible}, \\ 0, & \text{si } M_{j,v} \text{ es incompatible}, \end{cases}$$

y $\operatorname{vol}(Q(v) \cap U(F)) = \sum_{j=1}^{K} |Q(v) \cap A_j|$*;*

(IV) rank *y* select *se calculan mediante conteos de prefijo y búsqueda binaria en tiempo polinómico en n y K.*

Demostración. (i) es la aditividad de volúmenes de una unión disjunta. (ii) es álgebra lineal: un punto de A_j es p_j más una combinación de los $b_{j,\ell}$. Para (iii), fijar las coordenadas de v impone una ecuación lineal por cada variable fijada sobre los coeficientes de la combinación. Si el sistema es incompatible, el conjunto de soluciones es vacío y la contribución es 0; no se sustituye por 2^{d_j}. Si es compatible, el conjunto de soluciones es una variedad afín de dimensión $d_j - r$, donde $r = \operatorname{rk}(M_{j,v})$, y tiene $2^{d_j - r}$ puntos. La suma sobre piezas es exacta porque son disjuntas. Para (iv), el autómata de prefijos del Lema 5.1 genera $O(n)$ patrones parciales v, uno por nivel y rama del autómata; para cada uno se aplica (iii), y las ramas incompatibles aportan cero. rank suma esos conteos y select desciende por los bits del prefijo usando los mismos conteos. Con eliminación gaussiana ingenua, un conteo de prefijo cuesta $O(n^3)$, de modo que rank cuesta $O(n^4 K)$; select por búsqueda binaria cuesta $O(n^5 K)$ y por descenso de prefijos $O(n^4 K)$. Con bitsets se reduce el factor polinómico. □

Observación 10.5 (El caso incompatible no es cosmético)**.** La corrección del punto (iii) cambia el valor, no solo la presentación. Si A_j es el subespacio $x_1 = 0$ en $n = 2$ y v fija $x_1 = 1$, entonces $Q(v) \cap A_j = \varnothing$; la fórmula que asigna rango 0 a un sistema incompatible devolvería $2^{d_j} = 2$, el volumen de la pieza, cuando el valor correcto es 0. Una versión anterior de este manuscrito cometía exactamente ese error; se detectó al comparar los conteos de prefijo con la enumeración completa. Por eso el protocolo experimental de la Sección 11 incluye intersecciones vacías, intersecciones de dimensión cero y sistemas compatibles de rango completo, y compara cada conteo con la tabla completa en $n \leq 10$.

Algoritmo 2 PrefixAffine(A_j, k)

Require: pieza afín $A_j = p_j + \langle b_{j,1}, \ldots, b_{j,d_j} \rangle$; entero k con n bits
Ensure: número de puntos de A_j con $\mathrm{pos}(x) < k$

```
c ← 0; v ← patrón vacío; tight ← true
for i = 1 to n do
  if tight then
    for b ∈ {0,1} con b < k_i do
      v' ← v ∪ {x_i = b}
      resolver M_{j,v'}; si es compatible, c ← c + 2^{d_j − rk(M_{j,v'})} {si no, suma 0}
    end for
    v' ← v ∪ {x_i = k_i}; si M_{j,v'} es compatible, v ← v'; si no, break
  end if
end for
return c
```

El Algoritmo 2 hace explícito el punto (iii): la compatibilidad se comprueba antes de sumar, y una rama incompatible aporta 0. El conteo total es la suma sobre las K piezas; $\mathrm{rank}_0(k) = k - \sum_j \textsc{PrefixAffine}(A_j, k)$ y select_0 desciende por los bits de k usando esos mismos conteos. La corrección del algoritmo es la del Teorema 10.4; su coste es $O(n)$ sistemas lineales por pieza, cada uno de coste $O(n^3)$ con eliminación gaussiana, de donde resultan las cotas $O(n^4 K)$ para rank y $O(n^4 K)$ para select por descenso.

Por qué rompe la cota de paridad. La paridad impar

$$O_n = \{x \in \Omega_n : x_1 \oplus \cdots \oplus x_n = 1\}$$

es un único subespacio afín de dimensión $n-1$: ADSOP $K = 1$. La DSOP mínima necesita 2^{n-1} cubos. La cláusula ancha también es un cubo, luego una pieza. La unión

$$U = \{x_0 = 0\} \cup \{x_0 = 1,\ \mathrm{paridad}(x_1, \ldots, x_{n-1}) = 1\}$$

es una ADSOP de 2 piezas, mientras que la DSOP mínima necesita $1 + 2^{n-2}$ cubos.

Dónde no ayuda. Para un conjunto genérico, la cápsula afín es todo el espacio y la ADSOP se degrada a la DSOP o a una partición peor. La Tabla 8 resume el experimento en $n = 12$. La columna DSOP es la salida de la heurística `cover`, no siempre el mínimo: para cubo+paridad el mínimo es $1 + 2^{n-2} = 1025$, mientras que `cover` devuelve 2048.

Familia	vol(U)	DSOP (`cover`)	ADSOP
paridad	2048	2048	1
cláusula ancha	64	1	1
cubo + paridad	3072	2048	2
subespacio afín aleatorio (dim. 6)	64	64	1
3-CNF aleatoria ($m = 2n$)	3969	627	627

Tabla 8: DSOP heurística frente a ADSOP en $n = 12$. La ADSOP comprime la paridad y los subespacios; no comprime una CNF aleatoria genérica.

ADSOP y Fourier. Los ejemplos siguientes muestran que ninguna de las dos representaciones domina a la otra en todas las familias. La paridad tiene ADSOP 1 y espectro $s = 2$. Una cláusula ancha de ancho w tiene ADSOP 1 y espectro $s = 2^w$. Un subespacio afín aleatorio de dimensión $n/2$ tiene ADSOP 1 y espectro $s = 2^{n/2}$. Para una función aleatoria no se espera compresión en

ninguna de las dos representaciones, y el experimento con 3-CNF aleatoria no muestra ganancia de la ADSOP. La compacidad depende, por tanto, de la estructura algebraica del conjunto prohibido.

10.4. Compilador ADSOP directo

La pregunta pendiente es cómo construir la ADSOP desde la CNF sin enumerar $U(F)$. El prototipo `sat_adsop_compiler.py` usa un solver SAT como oráculo y álgebra lineal sobre $\mathbb{F}_2$. La pieza central es la cápsula afín de un conjunto de puntos de $U(F)$.

Encontrar puntos de $U(F)$. Para una asignación parcial v, se codifica $U(F)$ como una DNF con selectores. Por cada cláusula C no satisfecha por v, se crea una variable selectora s_C y, por cada literal no fijado $\ell \in C$, la cláusula $(\neg s_C \vee \neg \ell)$. La cláusula grande $\bigvee_C s_C$ exige que al menos una cláusula sea falsificada. Un modelo del sistema es un punto de $U(F)$ que extiende v. Añadiendo una ecuación lineal $a \cdot x = b$ se restringe la búsqueda a un subespacio afín.

Lema 10.6 (Codificación de $U(F|_v)$)**.** *Sea $\mathcal{S}(F, v)$ la CNF formada por las unidades de v, las cláusulas $(\neg s_C \vee \neg \ell)$ para cada cláusula C no satisfecha por v y cada literal no fijado $\ell \in C$, y la cláusula $\bigvee_C s_C$. Si no hay selectores, la cláusula grande es vacía y el sistema es insatisfacible. Entonces $\mathcal{S}(F, v)$ es satisfacible si y solo si $U(F|_v) \neq \emptyset$. Además, la proyección de todo modelo sobre las variables originales pertenece a $U(F|_v)$, y todo punto de $U(F|_v)$ se extiende a un modelo.*

Demostración. Si $x \in U(F|_v)$, alguna cláusula C es falsificada por x. Se toma $s_C = 1$ y $s_{C'} = 0$ para $C' \neq C$. Las unidades de v se satisfacen porque x extiende v; las cláusulas $(\neg s_C \vee \neg \ell)$ se satisfacen porque x hace falsos los literales no fijados de C; las cláusulas con $s_{C'} = 0$ se satisfacen por $\neg s_{C'}$; y la cláusula grande se satisface por s_C. Recíprocamente, si hay un modelo, la cláusula grande fuerza $s_C = 1$ para alguna C. Las cláusulas $(\neg s_C \vee \neg \ell)$ fuerzan entonces a que todos los literales no fijados de C sean falsos; los literales fijados son falsos porque C no está satisfecha por v. Por tanto C es falsificada y la proyección del modelo pertenece a $U(F|_v)$. La última afirmación se sigue de la primera construcción. □

Con este lema, FindU(F, v) se implementa resolviendo $\mathcal{S}(F, v)$ y proyectando un modelo sobre las variables originales; si el sistema es insatisfacible, devuelve $\perp$.

Decidir si $U(F)$ es afín. Sea H la cápsula afín de los puntos encontrados. Si $F|_v \wedge [x \in H]$ es satisfacible, H contiene un punto que satisface F y, por tanto, $U(F|_v)$ no es afín. Si no lo es, $H \subseteq U(F|_v)$. Las ecuaciones de H son una base del complemento ortogonal de su espacio de direcciones; se calculan por eliminación gaussiana. Para comprobar la otra inclusión, se recorre esa base: para cada ecuación $a \cdot x = b$, se busca un punto de $U(F|_v)$ con $a \cdot x = 1 - b$. Si no existe ninguno, todo punto de $U(F|_v)$ satisface todas las ecuaciones y $U(F|_v) \subseteq H$; junto con $H \subseteq U(F|_v)$, se concluye $U(F|_v) = H$. Si existe, se añade el punto a la cápsula y se repite. Cada iteración aumenta la dimensión afín en al menos uno, de modo que el proceso termina en a lo sumo $n + 1$ iteraciones.

Algoritmo 3 AffineClosure(F, v)

Require: CNF F, asignación parcial v

Ensure: EMPTY, AFFINE con H, o NOT-AFFINE con un contraejemplo (o un error de terminación, caso inalcanzable)

```
p ← FindU(F, v)
if p = ⊥ then
  return EMPTY
end if
P ← {p}
for iter = 1, ..., n + 1 do
  H ← AffineHull(P)
  c ← modelo de F|_v ∧ [x ∈ H]
  if c ≠ ⊥ then
    return NOT-AFFINE con c y P
  end if
  q ← ⊥
  for cada ecuación a · x = b de H do
    q ← FindU(F, v ∧ [a · x = 1 − b])
    if q ≠ ⊥ then
      break
    end if
  end for
  if q = ⊥ then
    return AFFINE con H y P
  end if
  P ← P ∪ {q}
end for
return NOT-AFFINE con un error de terminación
```

Lema 10.7 (Crecimiento estricto de la cápsula)**.** *En una iteración de AffineClosure, sea $H = \mathit{AffineHull}(P)$ y sea $q \in U(F|_v)$ un punto que viola alguna ecuación de H. Entonces*

$$\dim \mathit{AffineHull}(P \cup \{q\}) > \dim H.$$

Demostración. Como $q \notin H$, la cápsula afín $H' = \text{AffineHull}(P \cup \{q\})$ contiene estrictamente a H. Si $\dim H' = \dim H$, entonces H' sería una variedad afín de la misma dimensión que H que contiene a H; dos variedades afines de la misma dimensión, una contenida en la otra, son iguales, contradicción. Por tanto $\dim H' \geq \dim H + 1$. □

Lema 10.8 (Corrección de AffineClosure)**.** *Bajo una asignación parcial v, AffineClosure devuelve EMPTY si y solo si $U(F|_v) = \emptyset$; devuelve AFFINE con H si y solo si $U(F|_v) = H$ es un subespacio afín; y devuelve NOT-AFFINE con un punto satisfaciente de H si $U(F|_v)$ no es afín.*

Demostración. Por el Lema 10.6, si $p = \bot$ la codificación DNF de $U(F|_v)$ es insatisfacible y $U(F|_v) = \emptyset$. En otro caso, H es la cápsula afín de puntos de $U(F|_v)$, luego H es afín y contiene a P. Si $F|_v \wedge [x \in H]$ es satisfacible, H contiene un punto fuera de $U(F|_v)$; como toda variedad afín que contiene a P contiene a H, $U(F|_v)$ no puede ser afín. Si es insatisfacible, $H \subseteq U(F|_v)$. Si además ningún punto de $U(F|_v)$ viola una ecuación de H, entonces $U(F|_v) \subseteq H$ y $U(F|_v) = H$. Si algún punto viola una ecuación, está fuera de H y se añade a P; por el Lema 10.7 la dimensión de la cápsula aumenta al menos uno en cada iteración, de modo que el bucle termina en a lo sumo $n + 1$ iteraciones. La rama de error no se alcanza: la dimensión afín es a lo sumo n. □

Algoritmo 4 CompileADSOP(F, v)

Require: CNF F, asignación parcial v (inicialmente vacía)
Ensure: una familia disjunta de subespacios afines que cubre $U(F|_v)$

```
if F|_v es UNSAT then
    return {Q(v)} {todo el cubo pertenece a U}
end if
(status, H) ← AffineClosure(F, v)
if status = EMPTY then
    return ∅
end if
if status = AFFINE then
    return {H}
end if
elegir una variable libre i
return CompileADSOP(F, v ∪ {x_i = 0}) ∪ CompileADSOP(F, v ∪ {x_i = 1})
```

La elección de la variable libre no afecta a la corrección del algoritmo. En el prototipo se elige una variable que separe los puntos de U encontrados durante la cápsula, con el fin de reducir el número de nodos; esta heurística no forma parte del teorema de corrección.

Teorema 10.9 (Corrección del compilador ADSOP)**.** *Las piezas que devuelve CompileADSOP son subespacios afines, están contenidas en $U(F|_v)$, son disjuntas dos a dos y cubren $U(F|_v)$.*

Demostración. Se induce en la profundidad de la recursión. Si $F|_v$ es UNSAT, todo el cubo $Q(v)$ está en $U(F|_v)$ y es afín. Si AffineClosure devuelve EMPTY, el Lema 10.8 da $U(F|_v) = \emptyset$ y la rama devuelve el conjunto vacío. Si devuelve AFFINE, el mismo lema da $H = U(F|_v)$, y $H \subseteq Q(v)$ porque todos los puntos de H extienden v. Si devuelve NOT-AFFINE, las dos ramas fijan la variable libre $x_i = 0$ y $x_i = 1$; sus cubos son disjuntos y cubren $Q(v)$, y por hipótesis de inducción cada rama cubre exactamente su parte de $U(F|_v)$. La unión es disjunta y cubre $U(F|_v)$. □

Proposición 10.10 (Coste del prototipo)**.** *Cada llamada a AffineClosure realiza $O(n^2)$ consultas SAT y termina en $O(n)$ iteraciones. El árbol de CompileADSOP puede tener hasta $2^{n+1} - 1$ nodos, de modo que el número total de consultas SAT es $O(n^2\, 2^n)$. El prototipo no tiene una cota polinómica en n y m.*

Demostración. En cada iteración se hace una consulta $F|_v \wedge [x \in H]$ y, como máximo, una consulta por cada una de las n ecuaciones de H; el número de iteraciones está acotado por $n+1$ porque, por el Lema 10.7, la dimensión de H crece estrictamente. El árbol fija una variable por nivel y puede llegar a 2^n hojas; multiplicando por las $O(n^2)$ consultas por nodo se obtiene la cota $O(n^2\, 2^n)$. Cada consulta SAT es una instancia NP; el prototipo no elimina esa dureza, solo evita enumerar $U(F)$. □

Las restricciones $x \in H$ se codifican como ecuaciones XOR. El prototipo las traduce a CNF con variables auxiliares; alternativamente, un solver con XOR nativo las acepta directamente. La codificación preserva la satisfacibilidad, que es lo único que necesita el compilador; no preserva el número de modelos, pero el compilador no cuenta modelos en esas consultas.

La Tabla 9 compara el prototipo con la cobertura afín bruta y con la DSOP heurística en instancias de $n \leq 10$. Todas las coberturas se validaron contra la tabla completa: las piezas son disjuntas y cubren exactamente $U(F)$.

Familia	n	m	vol(U)	ADSOP directo	ADSOP bruto	DSOP `cover`	SAT	nodos
paridad	6	32	32	1	1	32	14	1
paridad	8	128	128	1	1	128	18	1
paridad	10	512	512	1	1	512	22	1
cláusula ancha	10	1	32	1	1	1	18	1
cubo+paridad	8	192	192	2	2	128	27	3
cubo+paridad	10	768	768	2	2	512	31	3
3-CNF $m = 2n$	8	16	225	45	88	92	629	89
3-CNF $m = 2n$	10	20	943	134	281	285	1804	267

Tabla 9: Compilador ADSOP directo. La columna ADSOP bruto usa la partición codiciosa con la lista completa de puntos; la columna DSOP `cover` es la heurística de cubos. El prototipo no usa `brute_force` para construir las piezas; la validación final sí compara con la tabla completa. La comparación es de calidad de la cobertura, no de coste: el compilador directo usa un oráculo SAT y puede ser exponencial.

Observación 10.11 (El prototipo produce menos piezas que la partición codiciosa)**.** En 3-CNF aleatoria el compilador directo produce menos piezas que la cobertura afín bruta (45 frente a 88 en $n = 8$, 134 frente a 281 en $n = 10$), porque la cápsula con SAT detecta una pieza afín grande antes de dividir. No es mínima; es una cota superior más ajustada. Las CNF de paridad y cubo+paridad son expansiones completas, con una cláusula por asignación de $U(F)$ ($m = 2^{n-1}$ y 192/768): el experimento mide la representación, no el tamaño de la entrada. Para cláusula ancha y 3-CNF aleatoria la entrada sí es compacta. La Tabla 11 separa la paridad directa de la paridad codificada con auxiliares; solo la primera tiene una ADSOP de una pieza por construcción, y la segunda debe medirse sobre la CNF codificada, no suponerse.

10.5. Límites y problemas abiertos

Las alternativas no cruzan la barrera #P. El conteo exacto sigue siendo #P-completo [14]; bajo #ETH (la variante de conteo de ETH) no hay algoritmo $2^{o(n)}$ para #SAT; el muestreo y el hashing dan aproximación, no exactitud; Karp–Luby estima $|U|$ pero la resta del complemento destruye la precisión relativa en el régimen raro. La ADSOP es exacta para familias afines, pero el prototipo usa un oráculo SAT y puede ser exponencial. Si existiera un compilador ADSOP uniforme polinómico para toda CNF, se seguiría #SAT $\in$ FP y, por Toda, PH = P; el prototipo no es ese compilador.

Problema abierto (Compilación ADSOP)**.** Caracterizar las familias de CNF que admiten una ADSOP de tamaño poly(n, m) y decidir si existe un compilador uniforme polinómico que la construya. La paridad muestra que la cota de DSOP no se hereda; un conjunto aleatorio sugiere que la partición afín también puede ser exponencial, pero no conocemos una cota inferior explícita para una familia de CNF de tamaño polinómico.

Problema abierto (Paridad compacta)**.** Resolver la paridad con una CNF compacta (con variables auxiliares y conteo existencial) en el prototipo ADSOP. La expansión completa usada en los experimentos tiene 2^{n-1} cláusulas; una CNF compacta requiere tratar la proyección sobre las variables originales.

Problema abierto (Puerta trasera afín)**.** Encontrar un conjunto pequeño B de variables tal que, al condicionar B, el residuo sea afín para toda asignación de B. Entonces

$$\#\mathsf{SAT}(F) = \sum_{\beta \in \{0,1\}^{|B|}} \#\mathsf{SAT}(F|_\beta)$$

y cada término se cuenta por álgebra lineal. El prototipo directo construye una puerta trasera sobre la marcha; la pregunta es si las CNF reales tienen una puerta trasera pequeña.

11. Protocolo experimental

El protocolo siguiente hace falsables las afirmaciones de ingeniería y permite comparar las tres ramas del motor.

11.1. Instancias

Familia	Parámetros	Qué mide
Cláusula ancha	$n \leq 24$, $w = 2, \ldots, n$	$K = 1$, consultas $O(n)$
Paridad	$n \leq 16$	$K = 2^{n-1}$, límite del motor DSOP
3-CNF aleatorias	$n \leq 20$, $m = 4.2n$	K típico y tiempo
PHP	p palomas, $p - 1$ nidos	coste de la prueba de insatisfacibilidad
Tseitin	grafos 3-regulares, carga impar	coste de la prueba y tamaño de la traza

Tabla 10: Familias del protocolo experimental.

11.2. Métricas

Para cada instancia se registran: n, m, #SAT, popcnt(S), K de COVERTRACE, tamaño de la ZDD (si se construye), bytes del certificado, tiempo de preprocesamiento, tiempo por consulta de bit, rank y select, y memoria. Para $n \leq 16$ se compara cada consulta con fuerza bruta; para $n \leq 24$ se compara al menos el conteo total con una implementación independiente.

11.3. Criterios de aceptación

(A1) Para $n \leq 8$, todos los valores de bit, rank y select coinciden con la fuerza bruta.
(A2) Para $n \leq 16$, #SAT coincide con el conteo exacto por enumeración.
(A3) En el rango $n \leq 16$, un certificado de DSOP corrupto es rechazado por el verificador; un certificado válido produce el mismo conteo que la fuerza bruta.
(A4) La rama de conteo, cuando está disponible, produce el mismo $\text{rank}_0(k)$ que la rama DSOP en las instancias donde ambas son viables.
(A5) El muestreo uniforme supera un test chi-cuadrado para $n \leq 12$.

11.4. Hipótesis experimentales

- Las cláusulas anchas producen K pequeño y consultas $O(n)$; la forma balanceada las fragmentaría en 2^{n-w} singletons.
- La paridad produce $K = 2^{n-1}$; el motor DSOP es exponencial en n y el oráculo de conteo (o una ZDD construida desde la ecuación lineal) es la alternativa.
- En 3-CNF aleatorias, K crece con n; el cruce entre DSOP y conteo depende de la instancia y debe medirse, no postularse.

11.5. Protocolo para las alternativas

Las alternativas de la Sección 10 se evalúan con dos protocolos independientes. El estocástico usa instancias con #SAT conocido para $n \leq 26$: solución única, cláusulas disjuntas, 3-CNF aleatorias y una instancia UNSAT. Se comparan el muestreo directo con 120 000 muestras, el hashing XOR con límite de 64 modelos y siete repeticiones, y Karp–Luby con 50 000 muestras. Se registran la estimación, el error relativo y el número de restricciones t (Tablas 6 y 7). El protocolo afín usa $n \leq 10$: se compila la ADSOP directa, se valida cada pieza contra la tabla completa y se comparan el número de piezas, las llamadas SAT y los nodos con la cobertura afín bruta y la DSOP heurística (Tabla 9).

Los criterios de aceptación son:

(A6) Toda cobertura ADSOP directa es disjunta y cubre exactamente $U(F)$ para $n \leq 10$.

(A7) La construcción de la ADSOP no usa la lista completa de puntos; la generación de las CNF de prueba y la validación final sí.

(A8) Las estimaciones estocásticas se comparan con el conteo exacto y se reporta el error, sin declarar exactitud.

(A9) El experimento de Karp–Luby reproduce la cancelación del complemento cuando $\#\mathsf{SAT}/2^n$ es minúsculo.

Las hipótesis son: la ADSOP directa comprime la paridad y el cubo+paridad; no comprime una 3-CNF aleatoria; el compilador directo puede producir menos piezas que la partición codiciosa porque la cápsula SAT detecta piezas grandes, pero no es un compilador polinómico.

11.6. Protocolo para el híbrido

La evaluación del híbrido no debe limitarse a mostrar que una rama gana en un benchmark. La Tabla 4 es el primer paso estructural; no sustituye la medición de tiempos ni de escalado. Para cada familia se registran el testigo o parámetro que activa la rama, el tiempo de construcción, el éxito o NoDisponible, el tiempo de las consultas y la corrección frente a fuerza bruta. La Tabla 11 fija la matriz mínima.

Familia	Generador	n	Rama / condición esperada
Ancha	una o pocas cláusulas de ancho $n/2$	10–40	DSOP con K pequeño.
Paridad directa	conjunto de asignaciones de paridad impar, sin auxiliares	6–20	ADSOP de una pieza; DSOP exponencial.
Paridad codificada	CNF compacta con auxiliares (Tseitin/XOR)	6–20	ADSOP real de la CNF codificada; no se asume una pieza.
Cubo+paridad	expansión por asignación	8–12	ADSOP 2; DSOP exponencial.
3-CNF aleatoria	$m = rn$, $r = 2, 3, 4, 4.2, 5$	10–30	Fuera de las condiciones; contador o NoDisponible.
Componentes	bloques disjuntos	10–30	DSOP o contador por componentes.
Treewidth	cadena y variantes de pathwidth acotado	10–40	Contador, DSOP o ZDD.
Backdoor afín	XOR/afín más una CNF pequeña	10–20	Contador afín o ADSOP.
Tseitin / PHP	generadores estándar	10–30	DSOP o contador; prueba de estrés.
Planted	3-CNF con solución plantada	20–50	Contador o estimación.
Real	DIMACS/SATLIB	variable	Mixta; se reporta la rama elegida.

Tabla 11: Matriz mínima de familias para la evaluación del híbrido. La columna de condición es una expectativa, no un resultado.

Las métricas estructurales son n, m, ancho de cláusula, componentes, una estimación de treewidth (por ejemplo, eliminación mínima), K de la DSOP, tamaño Z de la ZDD, sparsity s del espectro, número A de piezas ADSOP y disponibilidad del contador A. Para las instancias con auxiliares se registran por separado el número de variables originales, el número total de variables, la codificación utilizada (Tseitin, XOR nativo, expansión completa) y la región exacta que se cubre. Las métricas de ejecución son tiempo de preprocesamiento, tiempo por consulta de bit, rank, select, $\#\mathsf{SAT}$ y muestreo, memoria, tasa de éxito y tasa de NoDisponible. Para $n \leq 16$ se compara cada consulta con fuerza bruta; para $n \leq 24$ se compara el conteo total con una implementación independiente; para las ramas estocásticas se reporta el error relativo y el intervalo de confianza.

Los controles son: fuerza bruta para n pequeño, `sharpSAT`, `c2d` o `Dsharp` como contadores y compiladores de conocimiento independientes, ApproxMC para la estimación, COVERTRACE para la DSOP, el prototipo ADSOP para la rama afín y un bitvector materializado con índice

sucinto para rank/select en $n \leq 16$. Cada familia se etiqueta como *polinómica demostrada*, *condicional*, *empírica* o NoDisponible; un ajuste polinómico en un benchmark no se presenta como una cota de complejidad. Los tiempos se reportan como trabajo y, si se mide paralelismo, también como profundidad y factor de trabajo; no se comparan tiempos de pared entre modelos de coste distintos.

El criterio de aceptación es: corrección exacta en las ramas exactas, error controlado en las estocásticas, ninguna rama declara éxito si no cubre $U(F)$, y reproducibilidad de los generadores y semillas. La evaluación debe mostrar tanto los regímenes donde el motor es competitivo como los que quedan fuera; un resultado exponencial en una familia no refuta otro régimen, y uno polinómico en un benchmark no demuestra una cota uniforme.

11.7. Tres escenarios de medición

El protocolo distingue tres escenarios que la Tabla 12 no separa por sí sola. El primero es el tiempo total de una sola consulta, incluyendo la compilación: es el escenario relevante cuando se consulta una fórmula una vez. El segundo es el tiempo amortizado para Q consultas sobre la misma fórmula:

$$T_{\text{total}}(Q) = T_{\text{setup}} + Q\,T_{\text{query}}, \qquad Q^* = \frac{T_{\text{setup}}}{T_{\text{Kerberos}} - T_{\text{query}}},$$

donde Q^* es el número de consultas a partir del cual la rama compilada compensa su coste. El tercero es el tiempo de rank y select frente a una línea base que implemente las mismas operaciones, por ejemplo un bitvector materializado con un índice sucinto [12, 13] para $n \leq 16$, o un recorrido lineal de $S(F)$ cuando la materialización no es viable.

La Tabla 12 es un piloto de una sola ejecución y no cumple el protocolo completo. Para una versión publicable se exigen al menos diez semillas por configuración, media y desviación estándar, la versión de cada programa, el hardware, los comandos exactos y los generadores. La comparación con Kerberos/BASILISK es útil porque separa el coste de compilación del de consulta, pero no sustituye la comparación con un compilador de conocimiento estándar (c2d, Dsharp o sharpSAT) ni con un índice de rank/select materializado. Esas comparaciones forman parte del protocolo y quedan pendientes. El piloto no registró el hardware ni las semillas; por eso no se presenta como reproducible. Es una limitación declarada, no un resultado que deba tomarse como una medición definitiva.

Pruebas unitarias de ADSOP. Además de las pruebas de cobertura, el protocolo incluye cuatro casos límite para la fórmula corregida del Teorema 10.4: (i) una pieza afín y un patrón v cuyo sistema $M_{j,v}$ es incompatible, que debe aportar 0; (ii) una intersección de dimensión cero, es decir, un sistema compatible de rango d_j, que debe aportar 1; (iii) una intersección compatible de rango intermedio, que debe aportar $2^{d_j - r}$; y (iv) una pieza de dimensión cero, que debe aportar 1 si el punto satisface el patrón y 0 si no. Cada caso se compara con la enumeración completa en $n \leq 10$. Sin estas pruebas, un error como el de la fórmula anterior —asignar el volumen completo a una intersección vacía— pasaría desapercibido en las familias donde todas las intersecciones son compatibles.

11.8. Resultados preliminares: DSOP frente a Kerberos

El baseline de esta evaluación es Kerberos: SLIME para SAT y BASILISK para #SAT. No se comparan otros solvers; la pregunta es si la capa DSOP del paper aporta algo frente al mejor motor disponible del proyecto. La comparación con c2d, Dsharp o sharpSAT se especifica en el protocolo (Sección 11.6) y queda pendiente; por eso las conclusiones de este piloto son locales a Kerberos y no una evaluación del estado del arte. Se usó `tools/bench_kerberos.cpp` para

SLIME/BASILISK y `tools/paper_vs_kerberos.py` para el prototipo DSOP. Los tiempos son de una ejecución, en un solo hilo, con `-O3`; no incluyen el arranque del proceso ni la construcción del engine, y no son una suite de competición.

El prototipo DSOP usa `brute_force` para enumerar $U(F)$ y `cover` para cubrirlo; **no es COVERTRACE**. Por tanto, la columna de setup mide un compilador naive, no el compilador que el paper propone. La comparación sirve para separar la capa de consulta del coste de compilación.

Familia	n	K	Setup (ms)	DSOP #SAT (μs)	Kerberos #SAT (ms)	rank (μs)	select (μs)
ancha	12	1	1.89	0.70	0.068	7.2	35.8
paridad	10	512	89.0	167.6	5.86	178.3	2644.8
paridad	12	2048	1288.6	716.3	72.87	732.7	13422.5
cubo+paridad	10	512	104.5	173.6	7.49	177.0	3090.7
3-CNF $r = 2.0$	12	627	46.6	250.1	0.222	201.0	7701.1
3-CNF $r = 3.0$	12	134	34.0	56.8	0.147	51.5	1904.5
3-CNF $r = 4.2$	12	40	25.3	18.1	0.143	20.2	605.6
cadena	12	1373	41.6	536.5	0.121	799.4	15045.7

Tabla 12: Motor DSOP del paper frente a Kerberos/BASILISK en $n \leq 12$. El setup DSOP es el prototipo naive (`brute_force`+`cover`), no COVERTRACE. La consulta DSOP es posterior al setup; Kerberos #SAT es una llamada a BASILISK.

Familia	n	m	#SAT	Tiempo (ms)	Nodos
ancha	30	1	1 073 709 056	0.08	30
solución única	20	20	1	0.04	1
disjunta	18	6	117 649	0.07	11
paridad	12	2048	2 048	80.49	4095
3-CNF $r = 3.0$	24	72	1 584	1.06	390
3-CNF $r = 4.2$	24	100	0	0.25	13

Tabla 13: Kerberos/BASILISK: conteo exacto y tiempo de pared. Las familias ancha, solución única, disjunta y paridad tienen valor analítico; las 3-CNF se compararon con una enumeración independiente en $n \leq 24$.

Los resultados separan dos regímenes. En una sola consulta #SAT, Kerberos/BASILISK gana en todos los casos: el setup DSOP naive cuesta entre 1.9 ms y 1.29 s, mientras que BASILISK cuenta en 0.07–72.9 ms. Después del setup, la consulta DSOP es mucho más barata: 0.7–716 μs frente a 0.07–72.9 ms de BASILISK. Para consultas repetidas, la DSOP gana en ancho, paridad y cubo+paridad; en 3-CNF aleatorias la ventaja es pequeña o nula, y en la cadena Kerberos es más rápido. La DSOP es la única rama que ofrece `rank` y `select`; Kerberos no los tiene de forma nativa, y el `select` DSOP cuesta entre 36 μs y 15 ms según K y n.

La conclusión práctica es la del híbrido: usar Kerberos/BASILISK para una consulta puntual o cuando el contador es eficiente, y usar la DSOP cuando ya está compilada y se van a hacer muchas consultas agregadas o de rango/selección. El cuello sigue siendo el compilador: el prototipo naive no compite con COVERTRACE, que es precisamente lo que falta implementar y medir. Esta comparación no incluye el algoritmo de Anthropic, que no tiene implementación práctica.

El punto de equilibrio Q^* se calcula directamente de la tabla. Para la cláusula ancha en $n = 12$, $Q^* \approx 28$ consultas; para paridad en $n = 10$ y $n = 12$, $Q^* \approx 16$ y $Q^* \approx 18$; para cubo+paridad en $n = 10$, $Q^* \approx 14$. En cambio, para 3-CNF con $r = 3.0$ en $n = 12$ hacen falta unas 377 consultas, y para $r = 4.2$ unas 203. En $r = 2.0$ y en la cadena el denominador es negativo: la consulta DSOP es más lenta que un conteo de Kerberos, de modo que la rama compilada no compensa nunca en esos casos. Estas cifras son de una sola ejecución y sirven para ilustrar el cálculo, no como una conclusión de escalado; el protocolo completo exige las repeticiones y las líneas base descritas en la Sección 11.7.

Prueba sobre CNF emitidos por Ariadna. Se añadió el parámetro `-dsop-count` a Ariadna. Con él, `-analyze-cnf ... -count-cnf -dsop-count` usa el motor híbrido: materializado si $n \leq N_0$, DSOP si $n \leq N_{\text{dsop}}$ y $K \leq K_{\max}$, y BASILISK como contador de respaldo. La Tabla 14 compara el tiempo total de pared con el contador clásico de Ariadna sobre CNF emitidos por `-emit-cnf` desde ejemplos de Turing.

CNF	n	m	Ariadna (ms)	`-dsop-count` (ms)	Aceleración	Rama
`n1`	88	252	28.9	29.0	1.00×	DSOP, $K = 1$
`n2`	1 531	4 560	343.3	317.8	1.08×	contador
`n3`	13 341	39 965	2 342.1	2 181.1	1.07×	contador
`n4`	83 593	250 695	22 134.6	23 281.1	0.95×	contador

Tabla 14: Tiempo total de pared de `ariadna -analyze-cnf ... -count-cnf`, incluyendo análisis y parseo, sobre CNF emitidos por Ariadna. La columna `-dsop-count` usa la política automática; n1 activa DSOP; n2–n4 superan N_{dsop} y usan BASILISK.

El resultado no es una victoria uniforme. En n1 la DSOP detecta un cubo único ($K = 1$) y cuenta en microsegundos, pero el tiempo total está dominado por el análisis del CNF y no mejora. En n2 y n3 el contador BASILISK es un 7–8 % más rápido que el contador clásico; en n4 es un 5 % más lento. Forzar DSOP en n2 tarda 7.2 s en el setup, muy por encima de BASILISK, por lo que la política automática lo evita. La conclusión es la esperada: en los CNF grandes y UNSAT de Ariadna, Kerberos/BASILISK es el motor práctico; la DSOP queda como una rama opt-in para instancias pequeñas/estructuradas y consultas repetidas. No se cambia el camino por defecto de Ariadna.

12. Trabajo relacionado

Ecuación SAT. La codificación de la tabla de verdad como un entero aparece en [1] para CNF balanceadas y en [2] para CNF arbitrarias mediante OR bit a bit. La nota [4] unifica esta representación con la compilación COVERTRACE y demuestra la identidad fundamental entre el peso de Hamming, la región prohibida y la suma de volúmenes. Este artículo añade la capa de consulta: bit, rank, select, #SAT y muestreo.

Compilación de conocimiento. La DSOP es una forma de compilación de conocimiento en el sentido de [9]; en la terminología de esa literatura es una DNF disjunta o particionada, y su estudio como forma de dos niveles tiene una historia propia [26, 25]. COVERTRACE [3] proporciona un compilador de subcubos disjuntos con conteo exacto y extracción de testigos; la cota de paridad 2^{n-1} muestra que la compresión por cubos tiene límites. Nuestro Teorema 4.3 identifica la DSOP con una DF-CNF, lo que conecta la compilación con la sintaxis de cláusulas. La implicación condicional del Teorema 7.1 es la misma que el Teorema 7.4 de [3]. Conviene decirlo sin rodeos: la codificación densa no es la novedad matemática del artículo; la novedad es la capa de consulta, la corrección del caso afín y el mapa de barreras.

Otras formas de conocimiento. La literatura de compilación de conocimiento contiene d-DNNF, SDD, OBDD y ZDD, con jerarquías de tratabilidad y *compilation maps* bien estudiados [9, 27, 28, 29, 30]. La DSOP ocupa un lugar restringido en ese mapa: es una forma de dos niveles, más débil que d-DNNF en expresividad pero con consultas de conteo y rango muy directas. La comparación honesta no es "DSOP frente a d-DNNF" en general, sino "DSOP con K pequeño frente a la mejor estructura para la familia". La Proposición 4.8 muestra que hay familias de CNF de tamaño polinómico para las que la DSOP no es competitiva; la pregunta es si alguna otra forma del mapa sí lo es.

BDD y ZDD. Los diagramas de decisión binarios [10] y los ZDD [11] son estructuras canónicas para funciones y familias de conjuntos, fijado el orden de variables. La Sección 9 los usa como optimización de consulta, no como sustituto de la DSOP.

Fourier y Walsh–Hadamard. La transformada de Walsh–Hadamard expresa la tabla de verdad en la base de caracteres [8]. Un espectro disperso es otra representación comprimida: la paridad tiene dos coeficientes, mientras que una cláusula ancha tiene 2^w. El algoritmo de [7] mejora la DFT densa en un factor polilogarítmico; para $N = 2^n$ sigue siendo exponencial en n. La compilación de un espectro disperso desde la CNF es un problema abierto.

Rank y select. Los bitvectors sucintos permiten rank y select en tiempo constante con $2^n + o(2^n)$ bits [12, 13]. La literatura posterior mejora el espacio y el tiempo con estructuras de Raman–Raman–Rao [31] y estudia los límites de la consulta sucinta con Pătraşcu–Viola [32]. Esa cota es excelente, pero requiere materializar 2^n bits. Nuestro motor DSOP evita la materialización a cambio de un coste $O(nK)$ por consulta; el oráculo de conteo evita ambas cosas a cambio de una llamada a un contador exacto.

Conteo de modelos. El conteo exacto de modelos es #P-completo en general [14]. La esparsificación de [16] es una de las herramientas que permite reducir CNF arbitrarias a instancias de ocurrencia acotada. La instanciación con [5, 6] se formaliza como la hipótesis H-ETH en el Apéndice A.

Conteo aproximado. El FPRAS de Karp–Luby para la unión de conjuntos [20] y la caracterización de Stockmeyer del conteo aproximado en $\mathsf{BPP}^{\mathsf{NP}}$ [21] son el marco clásico. Para problemas auto-reducibles, el muestreo casi uniforme y el conteo aproximado son equivalentes [22]; esto explica por qué una muestra uniforme de soluciones no da el conteo exacto. ApproxMC [23] instancia el hashing XOR con un solver SAT. La Sección 10.2 usa estas ideas y muestra la cancelación del complemento: Karp–Luby estima $|U|$, no #SAT.

Subespacios afines y compilación. La ADSOP generaliza la DSOP de cubos a subespacios afines sobre $\mathbb{F}_2$; la paridad, que es el testigo negativo de la DSOP, se convierte en una sola pieza. La idea se sitúa en el mapa de compilación de conocimiento de Darwiche–Marquis [9]: se cambia el lenguaje de representación para ganar concisión en familias afines, a costa de un compilador más difícil. El prototipo de la Sección 10.4 usa un solver SAT y álgebra lineal; no es un compilador polinómico.

Complejidad. La NP-completitud de SAT es clásica [17, 18]. La relación entre el motor y P frente a NP se analiza en la Sección 7: el motor no proporciona un algoritmo uniforme polinómico para #SAT, y las estructuras que lo harían posible implicarían $\mathsf{PH} = \mathsf{P}$ por el teorema de Toda [15].

Complejidad de pruebas. El tamaño K de una DSOP puede leerse como la medida de una refutación en un sistema de cubos disjuntos. La comparación con resolución, *cutting planes* y *polynomial calculus* se apoya en la literatura clásica [33, 34]. La Proposición 4.8 es una cota inferior para ese sistema en una familia concreta; una teoría más general de cotas inferiores para DSOP y ADSOP es parte del programa de la Sección 7.14.

13. Discusión y problemas abiertos

13.1. Resultados incondicionales

Los resultados incondicionales son la equivalencia exacta entre coberturas disjuntas de $U(F)$ y DF-CNF equivalentes a F (Teorema 4.3); el motor rank–select con coste $O(nK)$ y $O(n^2K)$ (Teorema 5.3); la reducción de $\text{rank}_0(k)$ a conteo exacto con el comparador de prefijos (Teorema 6.2); la corrección del motor híbrido (Teorema 8.1); las consultas exactas sobre una ADSOP, con la corrección del caso incompatible (Teorema 10.4); y la separación de la ruta DSOP (Proposición 4.8). A ellos se añade la validación del compilador ADSOP directo de la Sección 10.4, que construye coberturas disjuntas sin enumerar la tabla en $n \leq 10$. La frontera de complejidad también es incondicional en su dirección condicional: el Teorema 7.1 demuestra que un compilador uniforme polinómico de DSOP implicaría $\mathsf{PH} = \mathsf{P}$, y la Sección 7 explica por qué el motor no proporciona ese compilador y qué rutas quedan abiertas.

El punto conceptual es que la Ecuación SAT no es solo un entero denso: su forma generalizada proporciona una interfaz de consulta. La DSOP comprime la región prohibida en cubos; el comparador de prefijos convierte una consulta de rango en una instancia de conteo; la búsqueda binaria convierte el conteo en select y muestreo.

13.2. El límite estructural

El tamaño K de la DSOP y el tamaño Z de la ZDD son los parámetros que gobiernan las consultas. La paridad muestra que K puede ser exponencial en n; la ZDD de paridad es $O(n)$ porque usa una representación lineal, pero construirla directamente desde la CNF es un problema abierto. La separación fuerte entre CNF de tamaño polinómico y DSOP de tamaño polinómico ya está establecida (Proposición 4.8); lo que no se demuestra aquí es la separación para ADSOP, ZDD o Fourier. El motor no elimina la dureza de contar modelos: la traslada a la compilación o al contador. La ADSOP desplaza el límite de los cubos a los subespacios afines, pero no elimina la compilación; los oráculos estocásticos desplazan la exactitud a la aproximación. La Sección 7 precisa esta afirmación con las rutas de la Tabla 1, las islas de tratabilidad y la condición exacta de colapso.

13.3. Problemas abiertos

Problema abierto (Compresión por cubos)**.** Caracterizar las familias de CNF para las cuales $\chi_{\sqcup}(U(F))$ es polinómico en n, m y, además, existe un compilador polinómico que produce la DSOP. La existencia de familias con $m = \text{poly}(n)$ y $\chi_{\sqcup} = 2^{\Omega(n)}$ ya no está abierta: la Proposición 4.8 la establece. Lo que queda abierto es la caracterización fina y la verificación de la cobertura.

Problema abierto (Separación afín)**.** Demostrar o refutar la Conjetura 7.5: ¿existe una familia de CNF con $m = \text{poly}(n)$ cuya ADSOP mínima sea $2^{\Omega(n)}$? La paridad muestra que la ADSOP puede comprimir lo que la DSOP no comprime; falta una cota inferior para una familia de tamaño polinómico. Una respuesta afirmativa cerraría la ruta afín como ruta universal. Una respuesta negativa no basta para $\mathsf{P} = \mathsf{NP}$: eliminar la cota exponencial no da una cota polinómica. Para el colapso haría falta la Conjetura 7.4, que es más fuerte.

Problema abierto (Decisión de tamaño mínimo)**.** Determinar la complejidad de decidir si $\chi_{\sqcup}(U(F)) \leq K$ para una CNF F y un entero K. La minimización absoluta de DSOP es NP-hard para funciones dadas por tabla de verdad [26]; la versión con una CNF como entrada y K como parámetro no está resuelta. Una reducción de SAT a esta decisión daría una conexión directa entre la compresión por cubos y NP.

Problema abierto (Compilación directa a ZDD)**.** Caracterizar las familias de CNF que admiten una ZDD de tamaño polinómico y construirla directamente, sin enumerar una DSOP. La paridad es un caso testigo: su ZDD es $O(n)$, pero su DSOP mínima es 2^{n-1}.

Problema abierto (Compresión de Fourier)**.** Caracterizar las familias de CNF cuyo bitvector $S(F)$ tiene espectro de Walsh–Hadamard de tamaño polinómico y construirlo en tiempo polinómico. La paridad tiene espectro de tamaño 2 y DSOP 2^{n-1}; una cláusula ancha de ancho w tiene espectro 2^w y DSOP 1. Si toda CNF tuviera un espectro polinómico construible en tiempo polinómico, se seguiría $\#\mathsf{SAT} \in \mathsf{FP}$ y $\mathsf{P} = \mathsf{NP}$.

Problema abierto (Tradeoff consulta–espacio)**.** Determinar si existe una estructura de tamaño $\mathrm{poly}(n, m)$ que responda rank en tiempo $\mathrm{polylog}(n)$ para familias de CNF con DSOP exponencial, o demostrar una cota inferior condicional. Si tal estructura se construyera en tiempo polinómico para toda CNF, se seguiría $\#\mathsf{SAT} \in \mathsf{FP}$ y, por Toda, $\mathsf{PH} = \mathsf{P}$.

Problema abierto (Verificación del oráculo)**.** Formalizar y verificar de forma independiente los algoritmos de [5, 6], o identificar el punto exacto que impide la cota subexponencial. El marco del Teorema 6.2 separa esa tarea de la capa de consulta.

Problema abierto (Verificación de cobertura)**.** El problema se formuló en la Sección 7.9: determinar si una DSOP de $U(F)$ puede verificarse en tiempo polinómico con una traza de construcción, y caracterizar las familias para las que la verificación sin traza es polinómica.

13.4. Alternativas y garantías

Las Secciones 10.2 y 10.3 no compiten con el motor DSOP en el mismo eje. Los oráculos estocásticos reducen el coste de la estimación a costa de renunciar a la exactitud: el muestreo directo falla en el régimen raro, el hashing XOR lo encuentra pero solo aproxima, y Karp–Luby estima $|U|$ con un FPRAS que la resta del complemento no convierte en un FPRAS de $\#\mathsf{SAT}$. La ADSOP proporciona exactitud para familias afines: la paridad pasa de 2^{n-1} cubos a una pieza y el cubo+paridad de $1+2^{n-2}$ cubos a dos. El coste es un compilador más difícil: el prototipo directo usa un oráculo SAT, hace $O(n^2)$ consultas por nodo y puede explorar un número exponencial de nodos. El experimento en $n \leq 10$ muestra que la construcción directa es viable y que en 3-CNF aleatoria produce menos piezas que la partición codiciosa bruta, pero no es un compilador polinómico. En ambos casos se cambia la representación y se traslada el coste; no se obtiene un algoritmo polinómico para $\#\mathsf{SAT}$.

13.5. Relación con la práctica

Slime y la comparación práctica. El solver Slime del repositorio Ariadna es un motor CDCL con HESS, CT y el backend de conteo Basilisk; es la base práctica para las consultas SAT y para la rama de contador. El algoritmo de Anthropic no es un solver CDCL, no publica implementación ni benchmarks y sus constantes son astronómicas; no compite con Slime en el régimen experimental. La comparación útil es Slime/CDCL frente al portafolio, no el algoritmo asintótico. Para un solver rápido, la palanca está en el motor CDCL, el preprocesamiento, la selección de ramas y la integración del contador; el resultado de Anthropic es una referencia teórica, no una implementación de producción.

El motor híbrido sugiere una arquitectura concreta para Ariadna y `satx`: una capa de representaciones con un verificador de certificados con traza, un motor rank–select sobre DSOP y un adaptador para contadores exactos. La función bit se responde siempre por evaluación; las consultas agregadas eligen la representación. Esta separación permite que un error en el contador no afecte a las consultas puntuales y que un certificado de DSOP se verifique de forma independiente. Para n pequeño, la cobertura se verifica por enumeración; para n mayor, la traza

de COVERTRACE es parte del certificado. La capa puede incluir una ADSOP y un estimador estocástico, pero el verificador debe distinguir una respuesta exacta de una aproximación con intervalo de confianza; esa distinción es parte del contrato de la interfaz.

14. Conclusiones

La Ecuación SAT codifica la tabla de verdad en un entero de 2^n bits. La consulta puntual de ese entero es directa; la consulta agregada es el problema real. Este artículo muestra que la forma generalizada, junto con la compilación COVERTRACE, produce un motor rank–select cuyo coste depende del tamaño de la DSOP, y que un contador exacto de modelos produce un oráculo de rank mediante la restricción de prefijo. La combinación de ambos es un motor híbrido que evita materializar la tabla y que usa cada representación en su régimen competitivo.

El resultado más sólido sigue siendo incondicional: la equivalencia entre DSOP y DF-CNF, el motor de consulta asociado y la reducción de rank a conteo. A ellos se añaden dos resultados que cambian el mapa. El primero es la corrección del teorema ADSOP: una intersección incompatible aporta cero, y el algoritmo de prefijos debe tratar ese caso; sin esa corrección, los conteos de rank y select sobre una ADSOP eran incorrectos. El segundo es la separación de la ruta DSOP: la construcción de Sasao, discutida por Bernasconi, Ciriani, Luccio y Pagli, da una familia de CNF con $O(n)$ cláusulas y DSOP mínima $2^{\Omega(n)}$. Eso no demuestra $\mathsf{P} \neq \mathsf{NP}$, pero descarta los cubos disjuntos como representación universal y convierte el programa en una pregunta sobre las demás islas. La ADSOP conserva la paridad en una pieza y responde consultas exactas; el prototipo directo valida la representación en $n \leq 10$, pero no es un compilador polinómico y su cota inferior sigue abierta. La ZDD y Fourier se presentan como compresiones condicionales, no universales. La parte subexponencial (Apéndice A) es condicional a H-ETH, anunciada por dos manuscritos pero no verificada, y se declara como tal. La evaluación preliminar frente a Kerberos/BASILISK muestra que Kerberos gana en una consulta #SAT y que el prototipo DSOP gana en consultas repetidas y `rank`/`select` una vez compilado; el compilador naive sigue siendo el cuello. En los CNF grandes de Ariadna la mejora no es uniforme (7–8 % en n2/n3, −5 % en n4), de modo que `-dsop-count` queda como opción y el camino por defecto no cambia. La Sección 7 responde a la pregunta de complejidad: el motor no implica $\mathsf{P} = \mathsf{NP}$ porque sus cotas polinómicas son condicionales al certificado o al contador. Pero la respuesta no es solo negativa: el artículo formula las islas de tratabilidad, demuestra que la isla DSOP no cubre todas las CNF y deja la ruta afín acotada por dos extremos, la universalidad y la separación, sin pretender que sean exhaustivos. La paralelización no cambia el diagnóstico: el trabajo total, no la profundidad, es el recurso que decide. La agenda abierta es ahora más precisa: cotas inferiores para ADSOP, puertas traseras afines, compilación directa a ZDD y Fourier, verificación de certificados y complejidad de pruebas.

Limitaciones. Este manuscrito no demuestra $\mathsf{P} = \mathsf{NP}$ ni $\mathsf{P} \neq \mathsf{NP}$. La cota inferior para DSOP se apoya en la construcción de Sasao y en la discusión de Bernasconi, Ciriani, Luccio y Pagli; no la reproducimos aquí. El compilador ADSOP es un prototipo con oráculo SAT, no un compilador polinómico. La evaluación experimental es un piloto de una sola ejecución. La parte H-ETH depende de manuscritos no verificados y se relega al Apéndice A. El programa hacia $\mathsf{P} = \mathsf{NP}$ es una agenda de investigación, no un resultado. El valor del artículo, si lo tiene, está en la corrección del caso afín, en la separación de la ruta DSOP y en el mapa de lo que faltaría; no en una solución del problema del milenio.

A. La instanciación H-ETH

Una nota condicional. Esta nota puede omitirse en una primera lectura: no interviene en los resultados de las Secciones 4–5 ni en el mapa de barreras. Se incluye porque los manuscritos de

2026 son la única instanciación subexponencial disponible y porque conviene separar lo que se sabe de lo que se anuncia.

Hipótesis A.1 (H-ETH)**.** Existe un algoritmo determinista que, para cada k fijo, cuenta las asignaciones satisfacientes de una k-CNF de n variables y m cláusulas en tiempo $2^{o(n)}\,\mathrm{poly}(n, m)$ y espacio polinómico. Los manuscritos de OpenAI [5] y Anthropic [6] anuncian algoritmos subexponenciales para SAT; H-ETH es la forma de conteo que se necesita en este artículo. H-ETH implica que ETH es falsa.

La hipótesis H-ETH es más fuerte que la negación de ETH para decisión: es la propiedad que necesita el corolario siguiente. Si un manuscrito solo refutara ETH para decisión, H-ETH quedaría como una hipótesis adicional y el corolario se leería bajo esa hipótesis.

Observación A.2 (Estado de la evidencia sobre H-ETH)**.** El 26 de octubre de 2026 OpenAI anunció públicamente una formalización en Lean de una refutación de ETH y un proceso de revisión externa dirigido por especialistas en complejidad [5]. Ese anuncio no equivale a una aceptación de la comunidad, y la distinción importa: refutar ETH no es demostrar $\mathsf{P} = \mathsf{NP}$, porque un algoritmo $2^{o(n)}$ puede seguir siendo superpolinomial. El manuscrito de Anthropic [6] afirma un algoritmo determinista $2^{o(n)}$ para $\#k$-SAT y una formalización en Lean 4; si es correcto, refuta ETH y SETH y da un contador de tiempo subexponencial. No declara una cota de espacio, y H-ETH exige espacio polinómico: sin ese análisis adicional no establece H-ETH en la forma del Corolario A.3. La afirmación es extraordinaria y no ha sido verificada por humanos: el apéndice del manuscrito indica que la prueba fue escrita y revisada por modelos, que el listado Lean contiene definiciones y que la prueba completa está en un repositorio separado, y que la constante no es explícita. El propio manuscrito señala que no ofrece algoritmos prácticos y que su mejora no aparece hasta $n > 10^{10\,000}$ en el caso $m = 5n$. No usamos la afirmación como establecida: el Corolario A.3 sigue siendo condicional a H-ETH. Si se confirma y el contador usa espacio polinómico, el corolario se vuelve incondicional para k-CNF, pero no se obtiene un algoritmo polinómico ni práctico.

Corolario A.3 (Bajo H-ETH)**.** *Si H-ETH es cierta, entonces para toda k-CNF F (de ancho fijo k) y todo prefijo $0 \le t \le 2^n$:*

(a) $\mathrm{rank}_0(t)$ *se calcula en* $2^{o(n)}\,\mathrm{poly}(n, m)$ *tiempo y espacio polinómico.*

(b) $\mathrm{select}_0(r)$ *y el muestreo uniforme se calculan en* $2^{o(n)}\,\mathrm{poly}(n, m)$ *tiempo y espacio polinómico.*

Demostración. Se aplica el Teorema 6.2 con el contador de H-ETH. El comparador añade $O(n)$ variables y $O(n)$ cláusulas; la CNF combinada tiene ancho acotado (a lo sumo $\max(k, 3)$) si se codifica con compuertas de aridad acotada y Tseitin, y H-ETH se aplica a ese ancho fijo. El número de consultas para select_0 es $O(n)$, y $n \cdot 2^{o(n)} = 2^{o(n)}$. □

Observación A.4 (Ancho arbitrario)**.** H-ETH está formulada para k-CNF. Para una CNF de ancho arbitrario hay que pagar una codificación a k-CNF (Tseitin o esparsificación); la cota resultante es en el número de variables codificadas, no necesariamente en las n originales. La reducción del comparador es parsimoniosa en el conteo, pero no en el ancho: preserva el número de modelos, no el ancho de las cláusulas de F. Por eso el Corolario A.3 se enuncia para k-CNF y no para CNF arbitrarias. Para una CNF con cláusulas de ancho no acotado, la ruta correcta es la DSOP o una representación que no pague la codificación. El motor DSOP de la Sección 5 no tiene esta restricción de ancho; por eso ambos métodos son complementarios.

Referencias

[1] O. Riveros. *A Unique Encoding of Satisfying Assignments for Balanced CNFs.* Manuscrito, 2025. Incluido en *Obras Completas*, Vol. I.

[2] O. Riveros. *A Generalization of the SAT Equation Theorem to Arbitrary CNF Formulas via Bitwise OR Encoding.* Manuscrito, 2025. Incluido en *Obras Completas*, Vol. I.

[3] O. Riveros. *COVERTRACE-SAT as Disjoint-Subcube Knowledge Compilation.* Documento unificado del Vol. I, 2026. Incluido en *Obras Completas*, Vol. I.

[4] O. Riveros. *Conexiones entre el Teorema de la Ecuación SAT y COVERTRACE-SAT.* Nota unificadora, 2026. Incluida en *Obras Completas*, Vol. I.

[5] OpenAI. *The Exponential Time Hypothesis is False.* Manuscrito, 17 de octubre de 2026 (revisado el 25 de octubre de 2026). `https://github.com/openai/math/blob/main/preprints/The-Exponential-Time-Hypothesis-is-False-October-25-2026/paper.pdf` Consultado el 27 de octubre de 2026.

[6] Claude and Anthropic. *A $2^{o(n)}$ time algorithm for 3-SAT.* Manuscrito, 26 de octubre de 2026.

[7] OpenAI. *An explicit power saving for the exact discrete Fourier transform.* Manuscrito, 25 de septiembre de 2026.

[8] R. O'Donnell. *Analysis of Boolean Functions.* Cambridge University Press, 2014.

[9] A. Darwiche and P. Marquis. A knowledge compilation map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002.

[10] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, 35(8):677–691, 1986.

[11] S. Minato. Zero-suppressed BDDs for set manipulation in combinatorial problems. In *Proceedings of the 30th Design Automation Conference*, pages 272–277, 1993.

[12] G. Jacobson. Space-efficient static trees and graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science*, pages 549–554, 1989.

[13] G. Navarro and E. Providel. Fast, small, simple rank/select on bitmaps. In *Proceedings of the 11th International Symposium on Experimental Algorithms*, pages 295–306, 2012.

[14] L. G. Valiant. The complexity of computing the permanent. *Theoretical Computer Science*, 8(2):189–201, 1979.

[15] S. Toda. PP is as hard as the polynomial-time hierarchy. *SIAM Journal on Computing*, 20(5):865–877, 1991.

[16] R. Impagliazzo, R. Paturi, and F. Zane. Which problems have strongly exponential complexity? *Journal of Computer and System Sciences*, 63(4):512–530, 2001.

[17] S. A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.

[18] L. A. Levin. Universal sequential search problems. *Problemy Peredachi Informatsii*, 9(3):115–116, 1973.

[19] S. Arora and B. Barak. *Computational Complexity: A Modern Approach.* Cambridge University Press, 2009.

[20] R. M. Karp and M. Luby. Monte-Carlo algorithms for enumeration and reliability problems. In *Proceedings of the 24th Annual Symposium on Foundations of Computer Science*, pages 56–64, 1983.

[21] L. Stockmeyer. The complexity of approximate counting. In *Proceedings of the 15th Annual ACM Symposium on Theory of Computing*, pages 118–126, 1983.

[22] M. Jerrum, L. Valiant, and V. Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theoretical Computer Science*, 43:169–188, 1986.

[23] S. Chakraborty, K. S. Meel, and M. Y. Vardi. A scalable approximate model counter. In *Principles and Practice of Constraint Programming*, pages 200–216, 2013.

[24] M. Mézard, G. Parisi, and R. Zecchina. Analytic and algorithmic solution of random satisfiability problems. *Science*, 297:812–815, 2002.

[25] T. Sasao. A design method for AND-OR-EXOR three-level networks. In *Proceedings of the ACM/IEEE International Workshop on Logic Synthesis*, pages 8:11–8:20, 1995.

[26] A. Bernasconi, V. Ciriani, F. Luccio, and L. Pagli. Compact DSOP and partial DSOP forms. *Theory of Computing Systems*, 53(4):583–608, 2013.

[27] A. Darwiche. Decomposable negation normal form. *Journal of the ACM*, 48(4):608–647, 2001.

[28] A. Darwiche. SDD: A new canonical representation of propositional knowledge bases. In *Proceedings of the 22nd International Joint Conference on Artificial Intelligence*, pages 819–826, 2011.

[29] J. Huang and A. Darwiche. DPLL with a trace: from SAT to knowledge compilation. In *Proceedings of the 19th International Joint Conference on Artificial Intelligence*, pages 156–162, 2005.

[30] T. Pipatsrisawat and A. Darwiche. A lower bound on the size of decomposable negation normal form. In *Proceedings of the 24th AAAI Conference on Artificial Intelligence*, pages 345–350, 2010.

[31] R. Raman, V. Raman, and S. S. Rao. Succinct indexable dictionaries with applications to encoding k-ary trees and multisets. In *Proceedings of the 13th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 233–242, 2002.

[32] M. Pătraşcu and E. Viola. Cell-probe lower bounds for succinct partial sums. In *Proceedings of the 21st Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 117–122, 2010.

[33] E. Ben-Sasson and A. Wigderson. Short proofs are narrow—resolution made simple. *Journal of the ACM*, 48(2):149–169, 2001.

[34] A. A. Razborov. Lower bounds on the monotone complexity of some Boolean functions. *Doklady Akademii Nauk SSSR*, 281:798–801, 1985.